INACAP

Área Informática y Telecomunicaciones

Ingeniería en Informática

Proyecto de Integración – TIHI63

Informe de Desarrollo

Spotly: plataforma web de arriendo temporal de espacios por hora, día o mes

Asignatura: Proyecto de Integración (TIHI63)

Sección: [completar]

Nombre del académico: [completar]

Integrantes:

Albert Ríos

[Integrante 2]

[Integrante 3]

Versión actualizada: 23 de octubre de 2026

Fecha de entrega: [dd/mm/2026]

Contenido

I. Introducción..... 3
II. Objetivos..... 4
Objetivo general..... 4
Objetivos específicos..... 4
III. Desarrollo..... 5
1. Diseño de la solución: topología, infraestructura y arquitectura..... 5
2. Análisis y selección de herramientas, lenguajes, hardware y servicios TI..... 8
3. Selección y evaluación de proveedores..... 13
4. Selección de plataformas y desarrollo del prototipo..... 15
5. Criterios de aceptación, estándares normativos y buenas prácticas..... 18
6. Planificación del proyecto: carta Gantt..... 22
IV. Conclusiones..... 25
Referencias..... 28
Anexo A. Capturas del prototipo..... 30

(Si el índice aparece vacío, en Word: clic derecho sobre él → Actualizar campos → Actualizar toda la tabla.)

I. Introducción

En la Unidad 1 del Proyecto de Integración, el equipo identificó un problema concreto del mercado chileno: los propietarios de espacios subutilizados (bodegas, estacionamientos, oficinas, salas de reuniones, locales y estudios) no contaban con un canal confiable para arrendarlos por períodos cortos, mientras que personas y pequeñas empresas que necesitaban un espacio por horas o días debían recurrir a avisos informales, sin verificación de identidad, sin contrato y sin un medio de pago seguro. A partir de ese diagnóstico se levantaron los requisitos y se definió **Spotly**, un marketplace web que conecta a propietarios y arrendatarios para arrendar espacios por hora, día o mes con verificación de identidad, contrato digital y pago en línea.

Durante la Unidad 2 el equipo pasó del análisis al desarrollo. Se diseñó la topología, la infraestructura y la arquitectura de la solución; se compararon de forma cualitativa y cuantitativa las tecnologías, lenguajes, plataformas, hardware y servicios TI disponibles; se evaluó a los proveedores en términos de costo, seguridad, calidad y rendimiento; y se construyó un prototipo funcional sobre React, TypeScript, Supabase (PostgreSQL), Vercel y Transbank Webpay, que implementó los procesos centrales del negocio: registro y verificación de usuarios, publicación de espacios, reserva sin sobreventa, contrato con huella SHA-256, pago, billetera de tarjetas, disputas y panel de administración.

En la etapa final de la unidad el prototipo se refinó a partir de la retroalimentación de usuarios: se incorporó una identidad visual propia (logotipo «sp●tly»), una portada con fotografías de espacios reales, animaciones con Motion y Anime.js, una pantalla que informa el avance del pago y su resultado (éxito, lista de espera o rechazo por saldo insuficiente), la cancelación de reservas por parte del propietario con reembolso total, un historial de reservas canceladas separado por quién canceló y paneles de indicadores que se pueden ver por día, semana o mes. Junto con este informe se entregan el Manual de usuario y el Manual de instalación y despliegue.

En el camino el proyecto enfrentó cambios relevantes respecto de la planificación inicial: la persistencia del prototipo se migró desde almacenamiento local del navegador a una base de datos PostgreSQL gestionada; la verificación de identidad automatizada mediante inteligencia artificial se reemplazó por una revisión manual por razones legales y de costo; se incorporó una billetera de tarjetas con Transbank Oneclick; y se agregó un sprint de mejoras de experiencia y gestión que no estaba en la línea base. Estos ajustes se registraron y justificaron en la carta Gantt.

El presente informe documentó ese trabajo siguiendo la estructura de la Guía ABPro 2. La sección de Desarrollo se organizó en seis apartados: diseño de la solución; análisis y selección de tecnologías; selección y evaluación de proveedores; plataformas y prototipo; criterios de aceptación, estándares y buenas prácticas; y planificación del proyecto. El informe cierra con las conclusiones del equipo y las referencias utilizadas.

II. Objetivos

Objetivo general

Desarrollar el diseño técnico y un prototipo funcional de Spotly, una plataforma web de arriendo temporal de espacios, seleccionando de manera fundamentada las tecnologías, proveedores y plataformas, y estableciendo los criterios de aceptación, estándares normativos y la planificación que aseguran una solución segura, de calidad y viable para el contexto chileno.

Objetivos específicos

1. Diseñar los diagramas de topología, infraestructura y arquitectura que describen los componentes de Spotly, sus conexiones y la distribución de responsabilidades entre capas.
2. Analizar cualitativa y cuantitativamente las herramientas, lenguajes de programación, hardware y servicios TI candidatos, y seleccionar los más adecuados mediante matrices de decisión ponderadas.
3. Seleccionar y evaluar a los proveedores de servicios (base de datos, hosting, pagos y control de versiones) considerando costo, seguridad, calidad, rendimiento y riesgo de dependencia.
4. Seleccionar las plataformas de desarrollo y ejecución, y construir un prototipo que implemente los procesos y subprocesos principales del negocio.
5. Definir criterios de aceptación verificables y alinear la solución con los estándares normativos y las buenas prácticas de rendimiento y seguridad aplicables.
6. Elaborar, revisar y justificar la carta Gantt del proyecto, identificando desviaciones respecto de la línea base y las acciones de ajuste.

III. Desarrollo

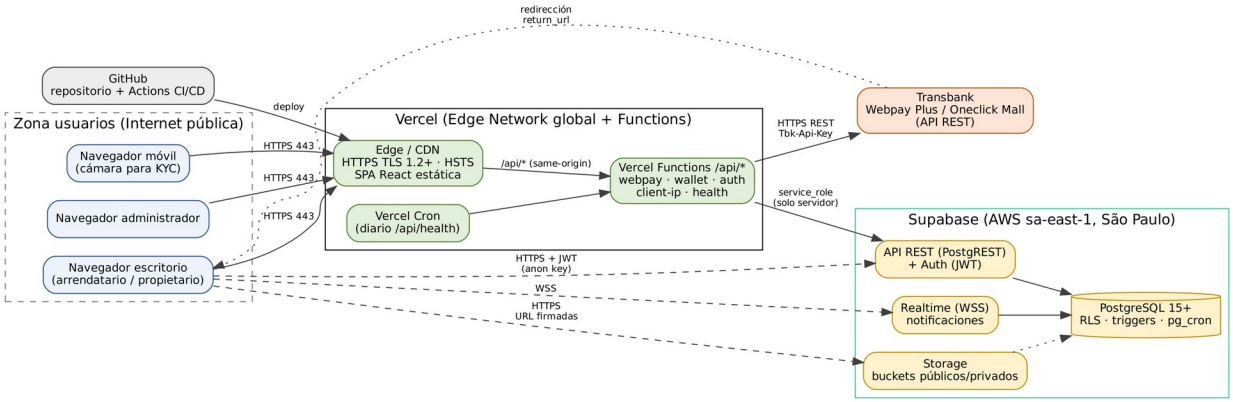
1. Diseño de la solución: topología, infraestructura y arquitectura

Spotly se diseña como una aplicación web de una sola página (SPA) desplegada en una red de distribución de contenidos, que consume servicios gestionados en la nube y funciones serverless para las operaciones sensibles. El diseño separa explícitamente lo que el navegador puede hacer con la llave pública (anon key) bajo políticas de seguridad a nivel de fila (RLS), de lo que solo el servidor puede hacer con credenciales privilegiadas (service_role y llaves de Transbank).

a) Diagrama de topología

La topología describe los nodos de la solución y los canales de comunicación entre ellos. Se trata de una topología en estrella lógica centrada en Internet, con tres zonas: la zona de usuarios (navegadores), la zona de aplicación (Vercel) y la zona de datos (Supabase), más el proveedor externo de pagos (Transbank).

Figura 1
Diagrama de topología de red de Spotly



Nota. Elaboración propia.

Los flujos principales son los siguientes:

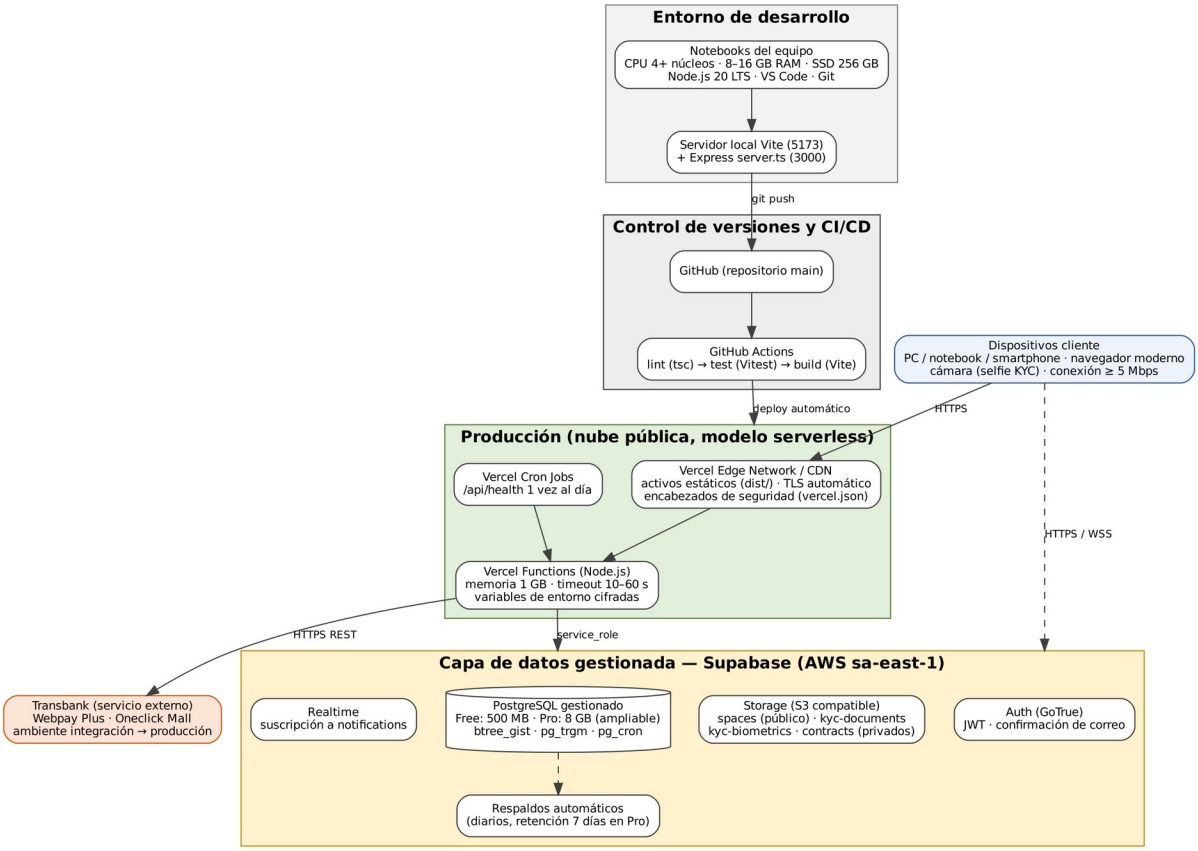
- **Usuario → Vercel Edge (HTTPS 443):** el navegador descarga la SPA desde el nodo de borde más cercano. La conexión usa TLS 1.2 o superior y cabeceras de seguridad (HSTS, X-Frame-Options DENY, X-Content-Type-Options nosniff, Referrer-Policy y Permissions-Policy).
- **Usuario → Supabase (HTTPS/WSS):** la SPA consulta directamente la API REST de PostgreSQL, Auth, Storage y Realtime usando el JWT del usuario. Cada consulta pasa por RLS, por lo que el usuario solo ve y modifica lo que le corresponde.
- **Usuario → /api/* → Transbank / Supabase:** las operaciones que requieren secretos (crear y confirmar pagos, inscribir tarjetas, reembolsos, inicio de sesión con bloqueo por intentos) pasan por funciones serverless, que son las únicas que conocen la llave service_role y las credenciales de comercio.
- **Transbank → Usuario (redirección):** tras el pago, Transbank redirige al navegador a la URL de retorno, y la función confirma (commit) la transacción del lado servidor antes de marcar la reserva como pagada.
- **Vercel Cron → /api/health:** una tarea diaria mantiene activo el proyecto de Supabase (el plan gratuito se pausa por inactividad) y verifica la salud del sistema.

- **GitHub → Vercel:** cada push a la rama principal ejecuta el pipeline de integración continua y despliega automáticamente.

b) Diagrama de infraestructura

La infraestructura adopta un modelo 100 % nube y serverless: el equipo no administra servidores ni sistemas operativos, sino recursos gestionados que escalan por demanda. Esto reduce el costo inicial a cero en la fase de prototipo y traslada el parcheo, los respaldos y la alta disponibilidad a los proveedores.

Figura 2
Diagrama de infraestructura de Spotly (desarrollo, CI/CD, producción y datos)



Nota. Elaboración propia.

Tabla 1
Recursos de infraestructura y su especificación

Recurso	Proveedor / servicio	Especificación	Función en Spotly
Hosting frontend	Vercel Edge Network	CDN global, TLS automático, compresión, caché inmutable de activos	Servir la SPA compilada (dist/) y aplicar cabeceras de seguridad
Cómputo backend	Vercel Functions (Node.js)	Ejecución bajo demanda, variables de entorno cifradas	webpay, wallet, auth, client-ip, health
Tareas programadas	Vercel Cron + pg_cron	Cron diario (Vercel) y cada 5 min (PostgreSQL)	Salud del sistema; expirar holds de pago y actualizar estados

Recurso	Proveedor / servicio	Especificación	Función en Spotly
Base de datos	Supabase PostgreSQL	Free 500 MB / Pro 8 GB; extensiones btree_gist, pg_trgm, pg_cron	Datos de dominio, reglas de negocio en triggers, RLS
Almacenamiento de objetos	Supabase Storage	Buckets: spaces (público); kyc-documents, kyc-biometrics, contracts (privados, URL firmadas de 1 h)	Fotos de espacios, documentos de identidad, contratos PDF
Autenticación	Supabase Auth	JWT, confirmación de correo, sesiones	Identidad de usuarios y roles
Tiempo real	Supabase Realtime	WebSocket seguro (WSS)	Notificaciones instantáneas
Pasarela de pago	Transbank	Webpay Plus y Oneclick Mall (API REST)	Cobros, tokenización de tarjetas y reembolsos
Repositorio y CI/CD	GitHub + GitHub Actions	Revisión de tipos (tsc), 88 pruebas (Vitest), build (Vite) y demo en GitHub Pages	Control de versiones y despliegue continuo
Estaciones de desarrollo	Notebooks del equipo	CPU 4+ núcleos, 8–16 GB RAM, SSD, Node.js 20 LTS	Desarrollo local y pruebas

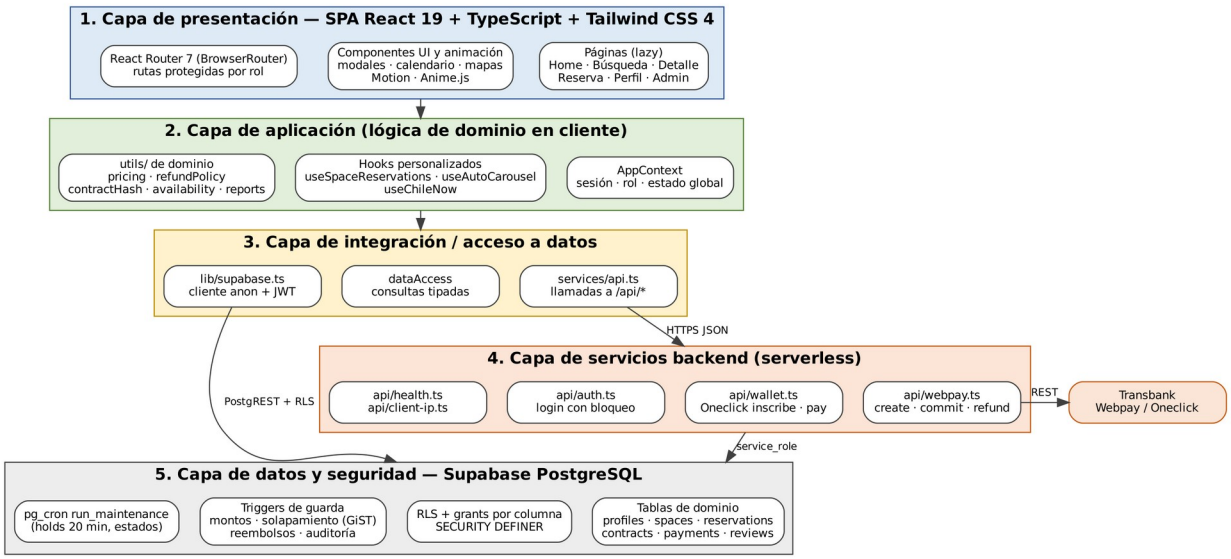
Nota. La región sugerida para Supabase es sa-east-1 (São Paulo), la más cercana a Chile entre las disponibles, lo que reduce la latencia de cada consulta.

c) Diagrama de arquitectura

La arquitectura de software sigue un estilo en capas con un backend como servicio (BaaS) y funciones serverless. La lógica de negocio crítica se implementa dos veces de forma coherente: en el cliente, para dar una respuesta inmediata al usuario (por ejemplo, calcular el precio), y en la base de datos, mediante triggers de guarda que validan lo mismo, de modo que un usuario que manipule el navegador no puede alterar montos, fechas ni estados.

Figura 3

Arquitectura lógica por capas de Spotly



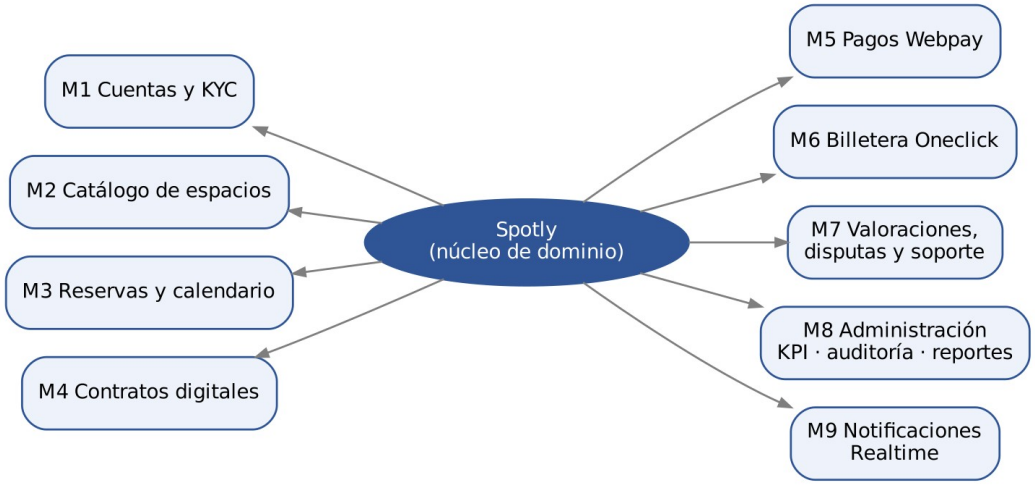
Nota. Elaboración propia.

Tabla 2
Responsabilidades por capa de la arquitectura

Capa	Componentes	Responsabilidad
1. Presentación	Páginas con carga diferida (lazy), componentes UI, React Router 7 (BrowserRouter), animaciones con Motion y Anime.js	Interfaz, navegación, validación de formularios, accesibilidad
2. Aplicación	AppContext, hooks (useSpaceReservations, useAutoCarousel, useChileNow), utils (pricing, refundPolicy, contractHash, availability, kpi, reports, cancellations)	Estado de sesión y reglas de dominio para respuesta inmediata
3. Integración	lib/supabase, dataAccess, services/api	Acceso tipado a datos y llamadas a la API propia
4. Servicios backend	Funciones /api/* en Vercel	Operaciones con secretos: pagos, billetera, login con bloqueo
5. Datos y seguridad	PostgreSQL con RLS, triggers, funciones SECURITY DEFINER, pg_cron, auditoría	Integridad, autorización fina, reglas de negocio no eludibles, trazabilidad

A nivel funcional, el sistema se divide en nueve módulos que comparten el núcleo de dominio (usuarios, espacios y reservas):

Figura 4
Módulos funcionales de Spotly



Nota. Elaboración propia.

2. Análisis y selección de herramientas, lenguajes, hardware y servicios TI

a) Metodología de evaluación

Para cada categoría tecnológica se identificaron al menos dos alternativas viables y se aplicaron dos análisis complementarios. El **análisis cualitativo** describe fortalezas, debilidades y riesgos de cada alternativa en el contexto de Spotly. El **análisis cuantitativo** usa una matriz de decisión ponderada: cada criterio recibe un peso (suma 100 %) según su importancia para el proyecto, cada alternativa se califica de 1 (muy deficiente) a 5 (excelente), y el puntaje ponderado es la suma de calificación × peso. Se elige la alternativa de mayor puntaje, siempre que no tenga una calificación 1 en un criterio crítico (seguridad o modelo de datos).

Los pesos se definieron a partir de las restricciones del proyecto: equipo pequeño con experiencia en JavaScript, presupuesto cero en la fase académica, plazo de un semestre, manejo de datos personales y de pago (alta exigencia de seguridad) y operación en Chile.

b) Análisis cualitativo

Tabla 3
Análisis cualitativo de las alternativas tecnológicas

Categoría	Alternativa	Fortalezas	Debilidades / riesgos
Frontend	React 19	Ecosistema más amplio; el equipo ya lo conoce; componentes reutilizables; carga diferida	Requiere elegir librerías complementarias (router, estado)
	Angular	Framework completo y opinado; TypeScript nativo	Curva de aprendizaje alta; más código repetitivo para un MVP
	Vue 3	Simple y liviano; buen rendimiento	Menor experiencia del equipo; ecosistema menor en Chile
Backend / BaaS	Supabase	PostgreSQL real, Auth, Storage, Realtime y RLS integrados; plan gratuito; región São Paulo	Plan gratuito se pausa por inactividad; dependencia parcial del proveedor
	Firebase	Muy rápido para prototipar; SDK maduro	Base NoSQL: difícil garantizar no solapamiento de reservas e integridad referencial
	Node/Express + AWS RDS	Control total; sin lock-in	Más infraestructura que operar; mayor costo y tiempo
Base de datos	PostgreSQL	ACID, restricciones de exclusión con rangos de tiempo, RLS, búsqueda trigram	Requiere diseño cuidadoso de políticas
	MySQL	Muy difundido y estable	Sin RLS nativo ni restricciones de exclusión por rango
	MongoDB	Esquema flexible	Transacciones y restricciones más débiles para reservas y pagos
Hosting	Vercel	Despliegue desde Git, funciones serverless, cron, CDN global	Plan Hobby solo para uso no comercial
	Netlify	Similar a Vercel en sitios estáticos	Funciones y cron menos integrados con el flujo actual
	Render	Ejecuta Express sin cambios	Instancias gratuitas se suspenden; arranque en frío lento
Pagos	Transbank Webpay	Estándar en Chile; débito, crédito y prepago; Oneclick para tarjetas guardadas; reembolsos por API	Proceso de afiliación comercial para producción
	Mercado Pago	Buena documentación y SDK	Menor penetración en pagos con débito chileno; comisiones mayores
	Flow	Agrega varios medios de pago	Capa intermedia adicional; menor reconocimiento de marca
Lenguaje	TypeScript 5.8	Tipado estático; errores detectados al compilar; mejor mantenibilidad	Curva inicial de tipos
	JavaScript	Sin paso de compilación	Errores en tiempo de ejecución; refactorizaciones riesgosas
Pruebas	Vitest	Nativo de Vite; rápido; compatible con API de Jest	Ecosistema más nuevo

Categoría	Alternativa	Fortalezas	Debilidades / riesgos
	Jest	Muy maduro	Configuración extra para ESM, TypeScript y Vite

c) Análisis cuantitativo (matrices ponderadas)

Tabla 4
Matriz ponderada: framework de frontend

Criterio	Peso	React (1–5)	Pond.	Angular (1–5)	Pond.	Vue (1–5)	Pond.
Experiencia del equipo / curva de aprendizaje	25 %	5	1,25	2	0,50	3	0,75
Ecosistema de librerías	20 %	5	1,00	4	0,80	4	0,80
Rendimiento en el navegador	15 %	4	0,60	4	0,60	5	0,75
Soporte de TypeScript	10 %	5	0,50	5	0,50	4	0,40
Comunidad y documentación	15 %	5	0,75	4	0,60	4	0,60
Despliegue como sitio estático	15 %	5	0,75	4	0,60	5	0,75
Total ponderado	**100 %**		**4,85**		**3,60**		**4,05**

Tabla 5
Matriz ponderada: plataforma de backend

Criterio	Peso	Supabase (1–5)	Pond.	Firebase (1–5)	Pond.	Node+AWS RDS (1–5)	Pond.
Costo inicial	20 %	5	1,00	4	0,80	2	0,40
Tiempo de desarrollo	25 %	5	1,25	5	1,25	2	0,50
Seguridad nativa (Auth, RLS)	20 %	5	1,00	4	0,80	3	0,60
Modelo relacional / SQL	15 %	5	0,75	1	0,15	5	0,75
Portabilidad (bajo lock-in)	10 %	4	0,40	2	0,20	5	0,50
Región y latencia hacia Chile	10 %	4	0,40	4	0,40	5	0,50
Total ponderado	**100 %**		**4,80**		**3,60**		**3,25**

Tabla 6
Matriz ponderada: motor de base de datos

Criterio	Peso	PostgreSQL (1–5)	Pond.	MySQL (1–5)	Pond.	MongoDB (1–5)	Pond.
Integridad transaccional (ACID)	25 %	5	1,25	5	1,25	3	0,75
Restricciones avanzadas (rangos, exclusión)	20 %	5	1,00	2	0,40	2	0,40
Seguridad a nivel de fila	20 %	5	1,00	2	0,40	2	0,40
Búsqueda de texto	10 %	4	0,40	3	0,30	4	0,40
Oferta gestionada en la nube	15 %	5	0,75	5	0,75	5	0,75
Costo	10 %	5	0,50	5	0,50	4	0,40
Total ponderado	**100 %**		**4,90**		**3,60**		**3,10**

Tabla 7
Matriz ponderada: hosting y despliegue

Criterio	Peso	Vercel (1–5)	Pond.	Netlify (1–5)	Pond.	Render (1–5)	Pond.
Costo	20 %	5	1,00	5	1,00	4	0,80
Integración con Git / CI	20 %	5	1,00	5	1,00	4	0,80
Funciones serverless	20 %	5	1,00	4	0,80	3	0,60
CDN y rendimiento	15 %	5	0,75	5	0,75	3	0,45
Seguridad (TLS, cabeceras)	15 %	5	0,75	5	0,75	4	0,60
Tareas programadas (cron)	10 %	4	0,40	3	0,30	4	0,40
Total ponderado	**100 %**		**4,90**		**4,60**		**3,65**

Tabla 8
Matriz ponderada: pasarela de pagos

Criterio	Peso	Webpay (1–5)	Pond.	Mercado Pago (1–5)	Pond.	Flow (1–5)	Pond.
Cobertura de medios de pago chilenos	25 %	5	1,25	4	1,00	4	1,00
Costo por transacción	15 %	4	0,60	3	0,45	4	0,60
Tokenización / cargos sin re-ingreso	15 %	5	0,75	4	0,60	3	0,45
Documentación y SDK	15 %	4	0,60	5	0,75	4	0,60
Confianza del usuario	15 %	5	0,75	4	0,60	3	0,45
Reembolsos por API	15 %	5	0,75	4	0,60	4	0,60
Total ponderado	**100 %**		**4,70**		**4,00**		**3,70**

Tabla 9
Matriz ponderada: lenguaje de programación

Criterio	Peso	TypeScript (1–5)	Pond.	JavaScript (1–5)	Pond.
Detección temprana de errores	30 %	5	1,50	2	0,60
Mantenibilidad	25 %	5	1,25	3	0,75
Productividad	20 %	4	0,80	4	0,80
Curva de aprendizaje	15 %	3	0,45	5	0,75
Ecosistema	10 %	5	0,50	5	0,50
Total ponderado	**100 %**		**4,50**		**3,40**

Tabla 10
Matriz ponderada: framework de pruebas

Criterio	Peso	Vitest (1–5)	Pond.	Jest (1–5)	Pond.
Integración con Vite	35 %	5	1,75	2	0,70
Velocidad de ejecución	25 %	5	1,25	3	0,75
Compatibilidad de API	20 %	4	0,80	5	1,00

Criterio	Peso	Vitest (1–5)	Pond.	Jest (1–5)	Pond.
Configuración TS / ESM	20 %	5	1,00	3	0,60
Total ponderado	**100 %**		**4,80**		**3,05**

Nota. Calificación de 1 (muy deficiente) a 5 (excelente). Pond. = calificación × peso. Las calificaciones se basan en la documentación oficial de cada tecnología, la experiencia del equipo durante el prototipo y las condiciones de los planes vigentes a octubre de 2026.

En todas las categorías la alternativa elegida obtuvo el mayor puntaje: React (4,85), Supabase (4,80), PostgreSQL (4,90), Vercel (4,90), Webpay (4,70), TypeScript (4,50) y Vitest (4,80). La diferencia más decisiva aparece en backend y base de datos: Firebase y MongoDB obtienen calificación 1–2 en criterios críticos (modelo relacional y restricciones), lo que las descarta para un sistema de reservas donde la sobreventa debe ser imposible.

d) Hardware y servicios TI

Al ser una solución serverless, Spotly no requiere adquirir servidores. El hardware se limita a las estaciones de desarrollo del equipo y a los requisitos mínimos de los dispositivos de los usuarios.

Tabla 11
Requisitos de hardware

Ámbito	Mínimo	Recomendado	Justificación
Estación de desarrollo	CPU 4 núcleos, 8 GB RAM, SSD 256 GB	CPU 6–8 núcleos, 16 GB RAM, SSD 512 GB	Vite, TypeScript y Vitest en paralelo con navegador y editor
Dispositivo del usuario	Navegador moderno (Chrome, Edge, Firefox, Safari, últimas 2 versiones), 4 GB RAM	Smartphone con cámara de 8 MP o más	La selfie y fotos de la cédula para KYC requieren cámara
Conectividad	5 Mbps	20 Mbps o más	Carga de fotos de espacios y documentos
Servidores	No aplica	No aplica	Cómputo y datos gestionados por Vercel y Supabase

Tabla 12
Servicios TI requeridos

Servicio	Solución	Fase
Dominio y DNS	Dominio .cl en NIC Chile apuntado a Vercel	Producción
Certificado TLS	Emitido y renovado automáticamente por Vercel	Prototipo y producción
Correo transaccional	SMTP de Supabase Auth en prototipo; SMTP propio (por ejemplo, Resend o Amazon SES) en producción por límites de envío	Prototipo → producción
Respaldos	Respaldos diarios de Supabase (Pro: 7 días de retención) + script SQL versionado en Git	Producción
Monitoreo	Registros de Vercel Functions, panel de Supabase y endpoint /api/health	Prototipo y producción
Documentación de API	Especificación OpenAPI (public/openapi.yaml)	Prototipo

e) Selección final y justificación

Tabla 13

Stack tecnológico seleccionado

Componente	Tecnología seleccionada	Justificación principal
Lenguaje	TypeScript 5.8	Tipado estático en frontend y backend; un solo lenguaje para todo el equipo
Frontend	React 19 + Vite 6 + Tailwind CSS 4	Mayor puntaje; experiencia del equipo; build rápido y CSS utilitario
Enrutamiento y UI	React Router 7, lucide-react	Rutas protegidas por rol; íconos livianos
Animación	Motion 12 y Anime.js 4	Transiciones entre vistas, gráficos animados y contadores; solo se animan transform y opacity para no afectar el rendimiento
Documentos	jsPDF + Web Crypto (SHA-256)	Contratos PDF y huella criptográfica sin servidor adicional
Backend	Supabase + Vercel Functions	Auth, RLS y Storage listos; funciones solo para operaciones con secretos
Base de datos	PostgreSQL (Supabase)	Restricción de exclusión anti-sobreventa, RLS, triggers y pg_cron
Pagos	Transbank Webpay Plus + Oneclick Mall	Estándar chileno; tokenización que evita manejar datos de tarjeta
Hosting y CI/CD	Vercel + GitHub Actions	Despliegue automático, cron, cabeceras de seguridad
Pruebas	Vitest	Integración nativa con Vite; cobertura de utilidades críticas

3. Selección y evaluación de proveedores

a) Proveedores candidatos y criterios

La selección de tecnologías determina un conjunto de proveedores de servicios. Para cada uno se evaluó si cumple las condiciones mínimas para operar Spotly con datos personales y pagos reales: costo sostenible en las fases de prototipo y producción, disponibilidad, certificaciones de seguridad, cercanía geográfica, soporte, calidad de la documentación y facilidad para migrar si el proveedor deja de ser conveniente. Un proveedor se aprueba si su puntaje ponderado es igual o superior a 3,5 y no tiene calificación 1 en seguridad.

b) Matriz de evaluación de proveedores

Tabla 14
Evaluación ponderada de los proveedores seleccionados

Criterio	Peso	Supabase (1–5)	Vercel (1–5)	Transbank (1–5)	GitHub (1–5)
Costo	20 %	5	5	4	5
Disponibilidad / SLA	15 %	3	4	5	4
Seguridad y certificaciones	20 %	4	4	5	5
Región / latencia	10 %	4	5	5	4
Soporte	10 %	3	3	4	4
Documentación	10 %	5	5	4	5
Portabilidad (lock-in)	15 %	5	4	3	4
Total ponderado	**100 %**	**4,20**	**4,30**	**4,30**	**4,50**

Nota. Umbral de aprobación: 3,5. Supabase obtiene 3 en SLA porque los planes Free y Pro no incluyen un acuerdo de nivel de servicio contractual (sí los planes Team y Enterprise); Transbank obtiene 3 en portabilidad porque la integración de pagos es específica de su API.

Los cuatro proveedores superan el umbral (Supabase 4,20, Vercel 4,30, Transbank 4,30, GitHub 4,50) y se aprueban. Las alternativas descartadas en el apartado anterior (Firebase, AWS RDS, Netlify, Render, Mercado Pago y Flow) se mantienen como proveedores de respaldo en caso de cambios de precio o de condiciones.

c) Costos estimados

Tabla 15
Costos mensuales estimados por fase

Proveedor	Fase prototipo (académica)	Fase producción (lanzamiento)	Observaciones
Supabase	Plan Free: US$ 0	Plan Pro: US$ 25/mes por organización	Pro incluye 8 GB de base de datos, 100.000 usuarios activos mensuales, respaldos diarios y crédito de cómputo de US$ 10
Vercel	Plan Hobby: US$ 0 (uso no comercial)	Plan Pro: US$ 20/mes por integrante con despliegue	El uso comercial exige plan Pro
Transbank	Ambiente de integración: US$ 0	Sin costo fijo mensual; comisión por transacción	Comisión según tipo de tarjeta y rubro (referencia pública Webpay: ~1,79 % + IVA en débito y ~2,79 % + IVA en crédito)
GitHub	Plan Free: US$ 0	Plan Free: US$ 0	Repositorio y Actions dentro de cuotas gratuitas
Dominio .cl	No requerido	≈ CLP 10.000 anuales	NIC Chile
Total fijo	**US$ 0**	**≈ US$ 45/mes (≈ CLP 43.000)**	**Tipo de cambio referencial de CLP 950 por dólar; las comisiones de pago son variables**

Nota. Precios vigentes publicados por cada proveedor a octubre de 2026; están sujetos a cambios y no incluyen IVA.

d) Seguridad, calidad y rendimiento de los proveedores

Tabla 16
Evaluación de seguridad, calidad y rendimiento por proveedor

Proveedor	Seguridad	Calidad	Rendimiento
Supabase	Cifrado en tránsito (TLS) y en reposo (AES-256); cumplimiento SOC 2 Tipo 2; RLS nativo; llaves separadas anon/service_role	PostgreSQL estándar; migraciones por SQL versionado; respaldos automáticos	Región sa-east-1; índices y conexión pooling; escalamiento de cómputo en planes pagados
Vercel	TLS automático; protección DDoS de la red; variables de entorno cifradas; cabeceras configurables	Despliegues inmutables con vista previa y rollback inmediato	CDN global con nodo en São Paulo; caché de activos; funciones bajo demanda
Transbank	Proveedor certificado PCI DSS; los datos de tarjeta nunca pasan por Spotly; autenticación 3-D Secure	Ambiente de integración con tarjetas de prueba; documentación y SDK oficiales	Plataforma nacional de alta disponibilidad; respuesta en segundos
GitHub	Autenticación de dos factores; secretos cifrados en Actions; alertas de dependencias (Dependabot)	Revisión de cambios por pull request; historial completo	Pipelines en ejecutores gestionados

e) Riesgos de dependencia y mitigación

Tabla 17
Riesgos asociados a los proveedores

Riesgo	Probabilidad	Impacto	Mitigación
Pausa del proyecto Supabase por inactividad (plan Free)	Alta	Medio	Vercel Cron diario a /api/health; plan Pro en producción
Aumento de precios o cambio de condiciones	Media	Medio	PostgreSQL estándar exportable con pg_dump; script SQL único versionado; alternativas evaluadas
Caída del proveedor de pagos	Baja	Alto	Holds de 20 min que liberan el espacio automáticamente; mensajes de reintento al usuario
Fuga de credenciales	Baja	Alto	Secretos solo en variables de entorno del servidor; rotación de llaves; nunca en el repositorio
Límites del plan gratuito (correo, almacenamiento)	Media	Bajo	SMTP propio y plan Pro antes del lanzamiento

4. Selección de plataformas y desarrollo del prototipo

a) Selección de plataformas

Se distinguen tres tipos de plataforma: la de desarrollo (donde el equipo construye), la de ejecución (donde corre la solución) y la de acceso (desde donde la usan las personas). La selección prioriza plataformas multiplataforma y gratuitas para el equipo, y la web como canal único para llegar a usuarios de escritorio y móvil sin publicar aplicaciones nativas.

Tabla 18
Plataformas seleccionadas

Tipo	Plataforma	Justificación
Desarrollo	Windows / macOS / Linux con Node.js 20 LTS, VS Code, Git	Herramientas gratuitas y multiplataforma; mismo entorno para todo el equipo
Construcción	Vite 6 (build) + TypeScript (tsc)	Compilación en segundos, división de código y carga diferida
Ejecución (frontend)	Vercel Edge Network; alternativa GitHub Pages (/spotly/ con HashRouter)	Sitio estático servido desde CDN; GitHub Pages sirve como demo gratuita
Ejecución (backend)	Vercel Functions; alternativa Render con Express (render.yaml)	Mismo código de API en ambos entornos
Datos	Supabase (PostgreSQL 15+)	Servicio gestionado con RLS y extensiones requeridas
Acceso de usuarios	Navegador web en escritorio y móvil (diseño responsive)	Sin instalación; un solo código para todos los dispositivos
Plataforma de pagos	Transbank (integración → producción)	Cambio de ambiente solo por variables de entorno

b) Descripción del prototipo

El prototipo de Spotly es una aplicación web funcional, no una maqueta. Implementa los flujos completos de arrendatario, propietario y administrador sobre datos reales en Supabase y pagos en el ambiente de integración de Transbank (tarjeta de prueba VISA 4051 8856 0044 6623). La

base de datos se crea con un único script transaccional (supabase/reset-and-setup.sql), lo que permite reproducir el entorno en cualquier proyecto nuevo.

El prototipo cuenta con tres roles: **arrendatario**, que busca, reserva, firma y paga; **propietario**, que publica espacios, gestiona su calendario y revisa sus ingresos; y **administrador**, que modera publicaciones, revisa identidades, resuelve disputas y monitorea indicadores.

c) Procesos y subprocesos implementados

Tabla 19
Procesos y subprocesos del prototipo

Proceso	Subprocesos	Reglas y controles clave
P1 Gestión de cuentas	P1.1 Registro con validación de mayoría de edad P1.2 Confirmación de correo P1.3 Inicio de sesión P1.4 Gestión de perfil y consentimientos	Solo mayores de 18 años; bloqueo tras 5 intentos fallidos por correo e IP (20 por IP) durante 15 min; registro de consentimientos
P2 Verificación de identidad (KYC)	P2.1 Carga de cédula (anverso y reverso) P2.2 Selfie P2.3 Certificado de antecedentes P2.4 Revisión manual del administrador	Documentos en buckets privados con URL firmadas de 1 h; sin aprobación no se puede reservar ni publicar
P3 Publicación de espacios	P3.1 Datos y tipo de espacio P3.2 Fotos, ubicación y precios por hora/día/mes P3.3 Envío a moderación P3.4 Aprobación o rechazo	Wizard de 3 pasos; RUT y dirección exacta del propietario son privados (grants por columna)
P4 Búsqueda y reserva	P4.1 Búsqueda con filtros P4.2 Detalle y calendario P4.3 Cálculo de monto P4.4 Creación de reserva con hold	Restricción de exclusión GiST impide solapamientos; el trigger recalcula el monto igual que pricing.ts
P5 Contrato digital	P5.1 Generación del contenido canónico P5.2 Huella SHA-256 P5.3 Firma electrónica simple P5.4 PDF descargable y control de integridad	Formato spotly-contract-v1; la huella se recalcula al abrir el contrato; Ley 19.799
P6 Pago y reembolso	P6.1 Pago Webpay Plus u Oneclick P6.2 Confirmación (commit) en servidor P6.3 Pantalla de progreso y resultado (éxito, lista de espera, rechazo) P6.4 Liquidación al propietario P6.5 Reembolso según política	Hold de 20 min; compute_refund replica refundPolicy.ts; tbk_user solo en servidor
P7 Gestión del propietario	P7.1 Panel de reservas y solicitudes P7.2 Calendario y bloqueos P7.3 Reserva inmediata o con aprobación P7.4 Cancelación con reembolso total P7.5 Indicadores por día, semana o mes P7.6 Reporte de ingresos (CSV)	Solo ve sus propios espacios (RLS); al cancelar, compute_refund devuelve el 100 %
P8 Post-arriendo	P8.1 Valoraciones P8.2 Disputas P8.3 Tickets de soporte P8.4 Notificaciones en tiempo real P8.5 Historial de canceladas por actor	Solo puede valorar quien completó una reserva; las anuladas por pago fallido no entran al historial

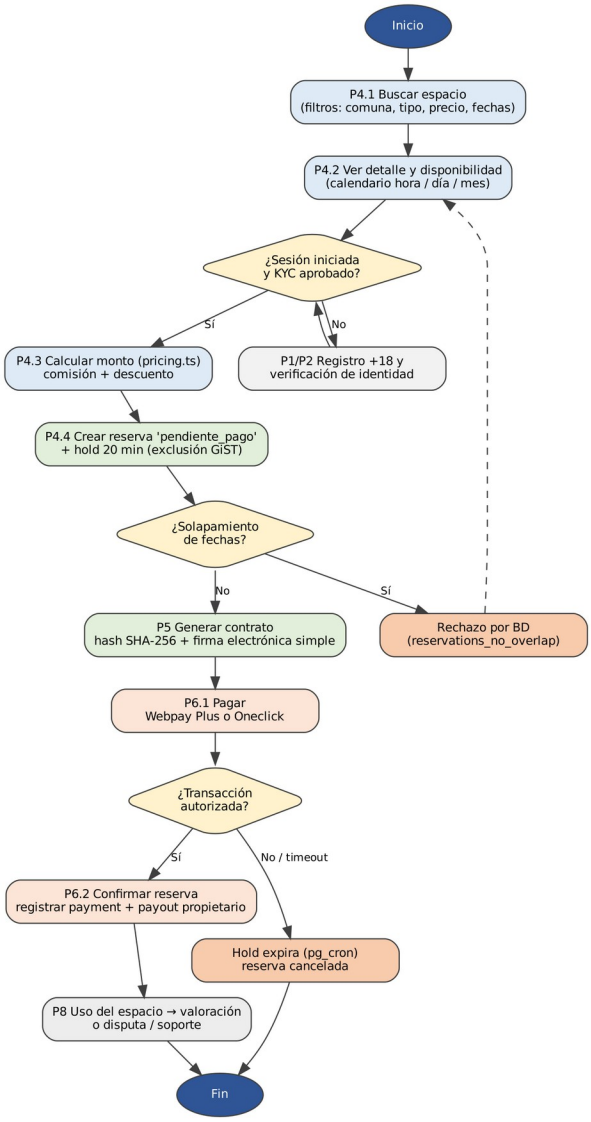
Proceso	Subprocesos	Reglas y controles clave
P9 Administración	P9.1 Moderación P9.2 Revisión KYC P9.3 KPI y reportes P9.4 Descuentos P9.5 Auditoría y anonimización	admin_kpis; audit_logs de solo inserción; admin_anonymize_user para derechos ARCO

d) Flujo del proceso principal

El proceso de mayor valor para el negocio es la reserva pagada, que encadena los procesos P4, P5 y P6. La figura muestra sus decisiones y los controles que impiden estados inconsistentes.

Figura 5

Diagrama de flujo del proceso principal: búsqueda, reserva, contrato y pago



Nota. Elaboración propia.

e) Pantallas del prototipo

Tabla 20
Vistas implementadas en el prototipo

Vista	Rol	Proceso asociado
Inicio y búsqueda con filtros	Público	P4.1
Detalle del espacio con calendario	Público / arrendatario	P4.2–P4.4
Registro e inicio de sesión	Público	P1
Verificación de identidad (cámara y documentos)	Arrendatario / propietario	P2
Publicar espacio (wizard de 3 pasos)	Propietario	P3
Checkout, firma de contrato y pago	Arrendatario	P5–P6
Billetera de tarjetas	Arrendatario	P6.1
Mis reservas, valoraciones y disputas	Arrendatario	P8
Panel del propietario e ingresos	Propietario	P7
Pantalla de progreso y resultado del pago	Arrendatario	P6.3
Historial de canceladas (arrendatario y propietario)	Arrendatario / propietario	P7.4, P8.5
Paneles con vista por día, semana o mes	Propietario / administrador	P7.5, P9.3
Panel de administración (KPI, KYC, moderación, auditoría)	Administrador	P9

Las capturas de pantalla de las vistas principales se presentan en el Anexo A; el Manual de usuario, entregado junto a este informe, describe cada una paso a paso.

5. Criterios de aceptación, estándares normativos y buenas prácticas

a) Criterios de aceptación

Cada criterio se redacta en formato verificable (dado / cuando / entonces) e indica cómo se comprueba. Un requisito se considera aceptado solo si su verificación resulta exitosa en el ambiente desplegado.

Tabla 21
Criterios de aceptación del prototipo

ID	Criterio	Verificación
CA-01	Dado un usuario menor de 18 años, cuando intenta registrarse, entonces el sistema rechaza el registro.	Prueba unitaria (ageValidation) + prueba manual
CA-02	Dados 5 intentos fallidos con el mismo correo e IP en 15 min, cuando se intenta un sexto, entonces el acceso se bloquea temporalmente.	Prueba manual contra /api/auth
CA-03	Dado un usuario sin KYC aprobado, cuando intenta reservar o publicar, entonces el sistema lo impide y lo dirige a verificación.	Prueba manual por rol
CA-04	Dado un espacio recién publicado, entonces no aparece en búsquedas hasta que un administrador lo aprueba.	Prueba manual
CA-05	Dadas dos reservas que se solapan para el mismo espacio, cuando se confirma la segunda, entonces la base de datos la rechaza.	Prueba de concurrencia (dos sesiones)
CA-06	Dado un monto alterado desde el navegador, cuando se crea la reserva, entonces el trigger lo rechaza o lo recalcula.	Prueba manual con herramientas de desarrollo + pruebas de pricing

ID	Criterio	Verificación
CA-07	Dada una reserva sin pago, cuando pasan 20 minutos, entonces se libera el espacio.	Verificación de pg_cron
CA-08	Dado un contrato firmado, cuando se modifica un carácter del contenido, entonces la verificación SHA-256 falla.	Prueba unitaria (contractHash) + visor del contrato
CA-09	Dado un pago autorizado por Transbank, entonces la reserva pasa a confirmada; si es rechazado, no.	Prueba en ambiente de integración
CA-10	Dada una cancelación, entonces el reembolso coincide con la política vigente según la antelación.	Pruebas unitarias (refundPolicy) + compute_refund
CA-11	El identificador de tarjeta de Oneclick (tbk_user) nunca se envía al navegador.	Inspección de red + prueba del API de billetera
CA-12	Un usuario no puede leer el RUT ni la dirección privada de otro propietario.	Consultas con JWT de otro usuario (RLS)
CA-13	Solo el rol administrador accede al panel de KPI, KYC y auditoría.	Prueba manual por rol + políticas RLS
CA-14	La página de inicio alcanza LCP ≤ 2,5 s y puntaje Lighthouse de rendimiento ≥ 85 en móvil.	Lighthouse / PageSpeed Insights
CA-15	Las vistas principales cumplen contraste y navegación por teclado (WCAG 2.1 AA).	Lighthouse accesibilidad ≥ 90 + revisión manual
CA-16	El pipeline de CI termina sin errores y la cobertura de utilidades críticas es ≥ 60 %.	GitHub Actions + reporte de Vitest
CA-17	Dado un pago en curso, entonces el usuario ve el avance (verificar, registrar contrato, pagar) y al terminar un resultado de éxito, lista de espera o rechazo.	Prueba manual + prueba en navegador automatizada (Playwright)
CA-18	Dada una reserva confirmada, cuando el propietario la cancela, entonces se reembolsa el 100 % y el horario queda libre.	Prueba unitaria (refundPolicy) + compute_refund
CA-19	Dada una reserva anulada por pago rechazado, entonces no aparece en el historial de canceladas; las canceladas por el arrendatario y por el propietario aparecen separadas.	Prueba unitaria (cancellations) + prueba manual
CA-20	Dado un espacio sin reserva inmediata, cuando se paga una reserva, entonces queda en lista de espera hasta que el propietario la acepta.	Prueba manual

b) Estándares normativos

Tabla 22
Normativa y estándares aplicables a Spotly

Norma / estándar	Exigencia relevante	Aplicación en Spotly	Estado
Ley 19.628 sobre protección de la vida privada	Tratamiento lícito de datos personales con consentimiento	Registro de consentimientos (user_consents); datos privados protegidos por RLS	Implementado
Ley 21.719 (protección de datos personales; vigente desde el 1 de diciembre de 2026)	Derechos ARCO y portabilidad; datos sensibles (biométricos); seguridad; notificación de brechas	Anonimización de usuarios; buckets privados para biometría; auditoría; revisión manual del KYC	Implementado parcialmente; faltan plazos de retención
Ley 19.799 sobre documentos y firma electrónica	Validez de la firma electrónica simple	Firma en SignatureCanvas + huella SHA-256 verificable	Implementado

Norma / estándar	Exigencia relevante	Aplicación en Spotly	Estado
Ley 18.101 de arrendamiento de predios urbanos	Contenido mínimo del contrato de arriendo	Plantilla de contrato con partes, objeto, plazo y precio	Implementado
Ley 19.496 de protección de los derechos de los consumidores	Información clara de precios, términos y derecho a reclamo	Desglose de precio antes de pagar; términos y condiciones; soporte y disputas	Implementado; faltan datos legales de la empresa
Reglamento de comercio electrónico (Decreto 6 de 2021, Ministerio de Economía)	Identificación del proveedor y confirmación de la transacción	Comprobante de reserva y pago	Parcial: datos del proveedor pendientes
PCI DSS v4.0	Protección de datos de tarjetas	Tokenización vía Transbank: Spotly no almacena ni procesa números de tarjeta	Alcance reducido
OWASP Top 10 (2021)	Mitigar riesgos web más comunes	Ver apartado e)	Implementado
ISO/IEC 27001:2022	Sistema de gestión de seguridad de la información	Controles de acceso, registro, respaldo y gestión de secretos alineados al anexo A	Referencia
ISO/IEC 25010	Modelo de calidad del producto de software	Criterios CA-01 a CA-20 cubren adecuación funcional, seguridad, rendimiento y usabilidad	Referencia
WCAG 2.1 nivel AA	Accesibilidad web	Contraste, textos alternativos, foco visible, navegación por teclado	En verificación
IEEE 830	Especificación de requisitos	Formato de los requisitos del Informe de Análisis	Aplicado en Unidad 1

c) Buenas prácticas de desarrollo

- **Control de versiones:** rama principal protegida, commits descriptivos y despliegue solo desde código que pasa el pipeline.
- **Integración continua:** GitHub Actions ejecuta verificación de tipos (tsc), pruebas (Vitest) y build en cada cambio.
- **Pruebas automatizadas:** 88 pruebas en 12 archivos que cubren el servicio de autenticación (/api/auth), la política de contraseñas, las cabeceras de seguridad, precios, política de reembolso, huella de contratos, disponibilidad horaria, reportes, historial de cancelaciones, contacto con el arrendatario y la API de billetera con mocks; umbral de cobertura del 60 % en utilidades críticas. Las vistas se revisan además en un navegador automatizado (Playwright) en escritorio y móvil.
- **Fuente única de verdad para reglas de negocio:** las funciones SQL (expected_reservation_amounts, compute_refund) replican exactamente pricing.ts y refundPolicy.ts, y las pruebas fijan su comportamiento.
- **Base de datos como código:** un único script SQL transaccional e idempotente crea tablas, políticas, triggers y tareas programadas.
- **Documentación de la API:** especificación OpenAPI versionada junto al código.
- **Configuración por entorno:** credenciales y ambiente de Transbank definidos por variables de entorno, nunca en el código.

d) Rendimiento

Tabla 23
Metas de rendimiento y método de verificación

Indicador	Meta	Técnica aplicada	Verificación
Largest Contentful Paint (LCP)	≤ 2,5 s (móvil 4G)	Carga diferida de páginas; activos en CDN con caché inmutable	Lighthouse
Interaction to Next Paint (INP)	≤ 200 ms	Componentes livianos; animaciones de Motion y Anime.js solo sobre transform y opacity	Lighthouse / Chrome DevTools
Cumulative Layout Shift (CLS)	≤ 0,1	Dimensiones reservadas para imágenes	Lighthouse
Respuesta de consultas de búsqueda	≤ 500 ms (p95)	Índices, pg_trgm, región sa-east-1, políticas RLS con (SELECT auth.uid())	Panel de Supabase (Query Performance)
Respuesta de funciones /api/*	≤ 1 s sin contar a Transbank	Funciones pequeñas y sin estado	Registros de Vercel
Disponibilidad del servicio	≥ 99,5 % mensual	Proveedores gestionados; health check diario	/api/health + monitoreo

Nota. Las metas se basan en los umbrales de Core Web Vitals definidos por Google. Los valores medidos deben registrarse sobre el despliegue final antes de la entrega.

e) Seguridad

Tabla 24
Controles de seguridad mapeados a OWASP Top 10 (2021)

Riesgo OWASP	Control implementado en Spotly
A01 Control de acceso roto	RLS en todas las tablas; grants por columna para datos privados; rutas protegidas por rol; funciones SECURITY DEFINER con search_path fijo
A02 Fallas criptográficas	TLS/HSTS en todo el tráfico; contraseñas guardadas solo como hash bcrypt con sal (Supabase Auth); SHA-256 para contratos; URL firmadas de 1 h para documentos privados; Spotly no almacena tarjetas
A03 Inyección	Consultas parametrizadas a través de PostgREST y del cliente de Supabase; validación de entradas
A04 Diseño inseguro	Reglas de negocio replicadas en triggers (montos, solapamiento, reembolsos) que el cliente no puede eludir
A05 Configuración de seguridad incorrecta	Content-Security-Policy estricta sin scripts en línea, COOP, CORP, HSTS y demás cabeceras generadas desde un único módulo (api/_security.ts) para vercel.json y server.ts; separación de llaves anon y service_role; variables de entorno
A06 Componentes vulnerables	Dependencias actualizadas; alertas de Dependabot; versiones fijadas en package-lock
A07 Fallas de identificación y autenticación	Verificación del correo con código de 6 dígitos (las cuentas sin verificar se eliminan a las 24 h); recuperación con enlace de un solo uso que cierra todas las sesiones; política de contraseñas en navegador y servidor; sesión en cookie HttpOnly + Secure + SameSite=Strict sin tokens en localStorage; protección anti-CSRF; bloqueo por intentos fallidos (correo + IP) en login, códigos y recuperación; KYC manual
A08 Fallas de integridad de software y datos	Despliegue solo desde CI; huella de contratos; montos validados en servidor
A09 Fallas de registro y monitoreo	audit_logs de solo inserción; registros de funciones; KPI de administración

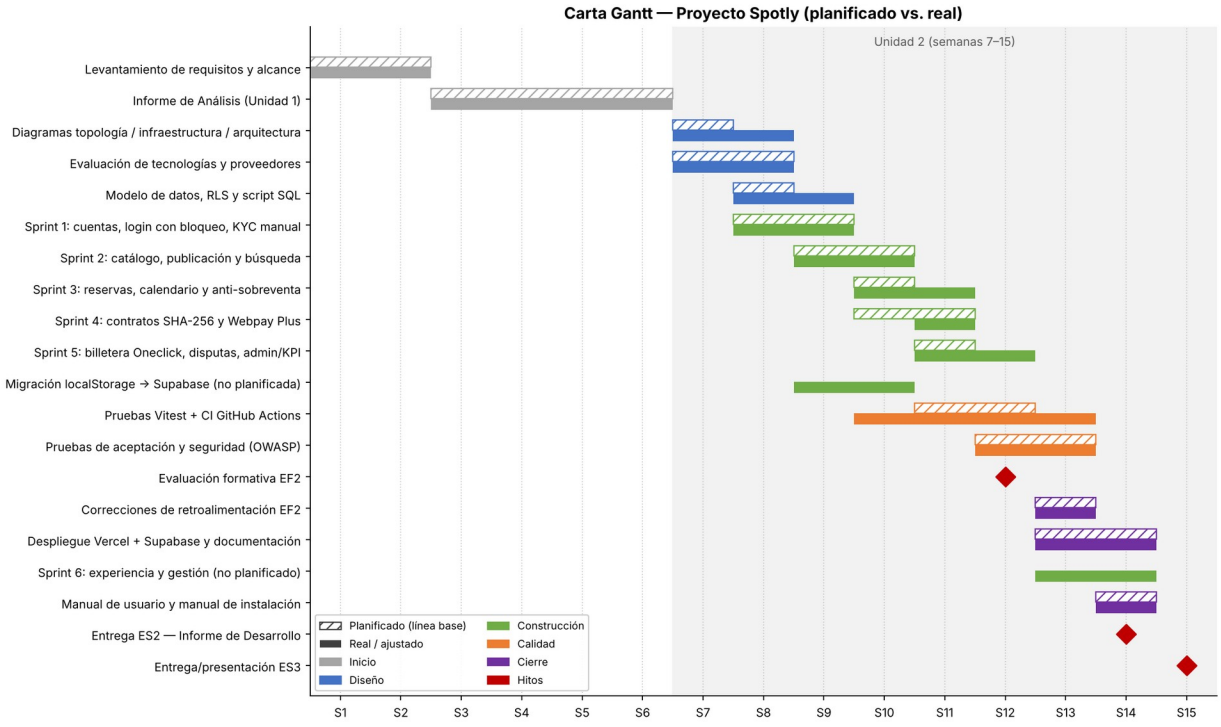
Riesgo OWASP	Control implementado en Spotly
A10 Falsificación de solicitudes del lado servidor (SSRF)	Las funciones solo llaman a dominios fijos de Transbank y Supabase

6. Planificación del proyecto: carta Gantt

a) Carta Gantt

La carta Gantt muestra las actividades del proyecto en semanas del semestre, comparando la línea base planificada al inicio de la Unidad 2 con la ejecución real. Las barras rayadas corresponden a lo planificado y las barras sólidas a lo ejecutado o ajustado; los rombos representan hitos de evaluación.

Figura 6
Carta Gantt del proyecto Spotly: línea base vs. ejecución real



Nota. Elaboración propia.

b) Detalle de actividades

Tabla 25
Actividades, responsables y semanas planificadas y reales

N°	Actividad	Semanas plan	Semanas real	Desviación	Entregable
1	Levantamiento de requisitos y alcance	S1–S2	S1–S2	0	Requisitos
2	Informe de Análisis (Unidad 1)	S3–S6	S3–S6	0	Informe U1
3	Diagramas de topología, infraestructura y arquitectura	S7	S7–S8	+1	Diagramas
4	Evaluación de tecnologías y proveedores	S7–S8	S7–S8	0	Matrices
5	Modelo de datos, RLS y script SQL	S8	S8–S9	+1	reset-and-setup.sql

N°	Actividad	Semanas plan	Semanas real	Desviación	Entregable
6	Sprint 1: cuentas, login con bloqueo, KYC	S8–S9	S8–S9	0	P1, P2
7	Sprint 2: catálogo, publicación y búsqueda	S9–S10	S9–S10	0	P3, P4.1
8	Migración localStorage → Supabase	—	S9–S10	No planificada	Persistencia real
9	Sprint 3: reservas, calendario, anti-sobreventa	S10	S10–S11	+1	P4
10	Sprint 4: contratos SHA-256 y Webpay Plus	S10–S11	S11	Compresión	P5, P6
11	Sprint 5: billetera Oneclick, disputas, admin/KPI	S11	S11–S12	+1	P6–P9
12	Pruebas Vitest + CI	S11–S12	S10–S13	Ampliada	Pipeline CI
13	Pruebas de aceptación y seguridad	S12–S13	S12–S13	0	CA-01 a CA-20
14	Hito: evaluación formativa EF2	S12	S12	0	Retroalimentación
15	Correcciones de retroalimentación	S13	S13	0	Versión corregida
16	Despliegue y documentación	S13–S14	S13–S14	0	Sistema desplegado
17	Sprint 6: experiencia y gestión (logo, animaciones, progreso de pago, cancelación del propietario, historial, paneles por período)	—	S13–S14	No planificada	Versión 2 del prototipo
18	Manual de usuario y manual de instalación	S14	S14	0	Manuales
19	Hito: entrega ES2 (Informe de Desarrollo)	S14	S14	0	Este informe
20	Hito: entrega ES3	S15	S15	0	Presentación

c) Revisión del cronograma y justificación de los ajustes

La revisión comparó semanalmente la ejecución con la línea base. Se detectaron cuatro desviaciones principales, todas absorbidas sin mover los hitos de evaluación:

- **Migración de la persistencia (no planificada, S9–S10):** el primer prototipo guardaba los datos en localStorage del navegador, lo que impedía compartir información entre usuarios, aplicar seguridad real y evitar la sobreventa. Se decidió migrar a Supabase. La actividad consumió dos semanas, pero eliminó una deuda técnica que habría invalidado los procesos de reserva y pago.
- **Cambio de KYC automatizado a revisión manual (S8–S9):** la verificación con inteligencia artificial implicaba enviar datos biométricos —datos sensibles según la Ley 21.719— a un tercero, con costo por uso y sin garantías de precisión. La revisión manual por un administrador redujo el riesgo legal y el costo, y simplificó el sprint 1.
- **Incorporación de la billetera Oneclick (S11–S12):** se agregó para permitir pagos sin reingresar la tarjeta, lo que mejora la conversión. Extendió el sprint 5 en una semana; se compensó comprimiendo el sprint 4, ya que la integración con Webpay Plus reutilizó la estructura de las funciones serverless.

- **Ampliación de las pruebas (S10–S13):** las pruebas se iniciaron una semana antes de lo planificado para cubrir las reglas de precio y reembolso antes de replicarlas en SQL, y se extendieron hasta la S13 para incorporar las observaciones de la EF2.
- **Sprint 6 de experiencia y gestión (no planificado, S13–S14):** las pruebas con usuarios mostraron tres problemas que afectaban la confianza: al pagar, la pantalla parecía quedarse cargando sin explicar qué ocurría; el propietario no podía cancelar una reserva confirmada; y los paneles solo mostraban datos mensuales. Se agregaron una pantalla de progreso y resultado del pago, la cancelación por el propietario con reembolso del 100 %, el historial de canceladas separado por quién canceló (excluyendo las anuladas por pago rechazado), la vista por día, semana o mes, y una identidad visual propia con animaciones. También se corrigió un error que impedía guardar la opción de reserva inmediata. Se probó un rediseño completo de las vistas, pero se descartó tras la revisión del equipo para conservar la interfaz ya validada; el esfuerzo se concentró en cambios puntuales. El sprint usó la holgura de la documentación, que se compensó generando los manuales a partir de capturas automatizadas.

El backend también se simplificó: en lugar de mantener un servidor Express permanente, la API se trasladó a funciones serverless de Vercel, conservando server.ts para desarrollo local y la alternativa de Render. Esto redujo el costo operativo y el tiempo de despliegue.

d) Ruta crítica y holguras

La ruta crítica del proyecto es: modelo de datos y RLS → reservas con anti-sobreventa → contratos y pagos → pruebas de aceptación → despliegue → entrega ES2. Cualquier atraso en estas actividades desplaza la entrega. Las actividades de documentación, diagramas y reportes del administrador tienen holgura de aproximadamente una semana, por lo que se usaron para absorber las desviaciones descritas.

e) Plan de acción hasta el cierre

Tabla 26
Acciones pendientes hasta la entrega final

Acción	Responsable	Plazo
Completar datos legales de la empresa (razón social, RUT, domicilio) en términos y contratos	Equipo	S14
Definir plazos de retención de documentos KYC y biometría (Ley 21.719)	Equipo	S14
Medir Lighthouse y tiempos de respuesta sobre el despliegue final y registrar resultados	Equipo	S14
Ejecutar la batería completa de criterios CA-01 a CA-20 y documentar evidencia	Equipo	S14
Ejecutar en Supabase la última versión de reset-and-setup.sql y redesplegar en Vercel	Equipo	S14
Preparar presentación y demostración en vivo para ES3	Equipo	S15

IV. Conclusiones

El desarrollo de Spotly durante la Unidad 2 permitió transformar un análisis de requisitos en una solución técnica concreta y funcional. El principal aprendizaje del equipo es que las decisiones de diseño tomadas al inicio —topología, infraestructura y arquitectura— condicionan todo lo que viene después. Separar con claridad lo que puede hacer el navegador de lo que solo puede hacer el servidor fue la decisión que más impacto tuvo en la seguridad del sistema: la SPA trabaja con la llave pública bajo políticas RLS, y las operaciones con secretos (pagos, billetera, inicio de sesión con bloqueo) viven exclusivamente en funciones serverless. Esta separación hace que, aun cuando un usuario manipule el código del navegador, no pueda acceder a datos de terceros ni alterar montos.

El análisis cualitativo y cuantitativo de tecnologías demostró su valor como herramienta de decisión. Más allá del puntaje final, las matrices ponderadas obligaron a explicitar qué era importante para el proyecto. El ejemplo más claro fue la elección de PostgreSQL sobre alternativas NoSQL: en un sistema de reservas, la imposibilidad de que dos personas reserven el mismo espacio en el mismo horario no puede depender de la lógica del cliente. La restricción de exclusión sobre rangos de tiempo de PostgreSQL resolvió ese problema en la propia base de datos, de manera atómica, algo que otras alternativas no ofrecían de forma nativa. Del mismo modo, TypeScript permitió detectar errores durante la compilación que en JavaScript habrían aparecido en producción.

La evaluación de proveedores mostró que es posible construir y operar un prototipo completo con costo fijo cero, y que el salto a producción es acotado: aproximadamente US$ 45 mensuales más las comisiones variables de Transbank. Sin embargo, también dejó en evidencia riesgos que no se ven en una comparación de precios, como la pausa por inactividad del plan gratuito de Supabase, la ausencia de un SLA contractual en los planes de entrada y la dependencia de la API específica del proveedor de pagos. Para cada uno se definió una mitigación concreta, y el uso de tecnologías estándar (PostgreSQL, TypeScript, HTTP) mantiene abierta la posibilidad de cambiar de proveedor si las condiciones cambian.

En cuanto al prototipo, el equipo logró implementar los nueve procesos definidos, desde el registro y la verificación de identidad hasta la administración con indicadores y auditoría. Un hallazgo importante fue la necesidad de duplicar de forma coherente las reglas de negocio críticas: el cálculo de precios y reembolsos existe en TypeScript para dar respuesta inmediata al usuario, y en SQL para que el servidor tenga la última palabra. Mantener ambas versiones sincronizadas exigió pruebas automatizadas que fijan el comportamiento esperado, lo que confirma que las pruebas no son un trámite final sino parte del diseño.

La incorporación de criterios de aceptación verificables y de un marco normativo explícito cambió la forma en que el equipo entiende la calidad. Revisar la Ley 21.719, que entra en vigencia el 1 de diciembre de 2026, llevó a reconsiderar la verificación de identidad automatizada con inteligencia artificial: los datos biométricos son datos sensibles y enviarlos a un tercero implicaba riesgos legales que el proyecto no podía asumir. La revisión manual es menos escalable, pero es la opción responsable para esta etapa. De igual forma, delegar el manejo de tarjetas en Transbank redujo drásticamente el alcance de PCI DSS, y alinear los controles con OWASP Top 10 entregó una lista de verificación concreta para la seguridad.

La gestión del cronograma fue otro aprendizaje relevante. La carta Gantt no se mantuvo idéntica a la línea base, y eso no fue una falla, sino el reflejo de decisiones técnicas fundamentadas: migrar la persistencia del almacenamiento local a una base de datos real, simplificar el backend a funciones serverless y agregar la billetera de tarjetas. Lo importante fue que cada desviación se

detectó a tiempo, se justificó y se compensó usando las holguras de actividades fuera de la ruta crítica, sin afectar los hitos de evaluación. Esto confirma la utilidad de revisar el cronograma semanalmente y no solo al final de cada etapa.

Desde la perspectiva del trabajo en equipo, el proyecto evidenció la importancia de contar con una fuente única de verdad. Mantener el esquema completo de la base de datos en un solo script transaccional y versionado permitió que cualquier integrante recreara el entorno desde cero en minutos, y evitó las inconsistencias típicas de modificar la base de datos manualmente desde un panel web. Lo mismo ocurrió con la especificación OpenAPI y con las variables de entorno documentadas: la documentación técnica dejó de ser un entregable aparte y pasó a formar parte del propio código, lo que redujo los errores de integración entre el frontend y las funciones serverless.

La integración con Transbank fue una de las experiencias más formativas. Trabajar con un proveedor de pagos real, aun en ambiente de integración, obligó al equipo a comprender el ciclo completo de una transacción: creación, redirección, confirmación en el servidor, manejo de rechazos, tiempos de espera y reembolsos. La decisión de confirmar siempre el pago desde el servidor, y no confiar en lo que informa el navegador al volver de Transbank, junto con los holds de 20 minutos que liberan automáticamente el espacio, demostró que la robustez de un sistema de pagos depende tanto del manejo de los casos de error como del flujo exitoso.

El equipo también valoró el enfoque de criterios de aceptación en formato dado / cuando / entonces. Redactar cada criterio de forma verificable obligó a precisar requisitos que en el Informe de Análisis estaban expresados de manera general, como «el sistema debe ser seguro» o «el sistema debe evitar dobles reservas». Al traducirlos a condiciones comprobables, con un método de verificación asociado, se hizo evidente qué parte de cada requisito ya estaba cubierta por pruebas automatizadas y qué parte dependía todavía de pruebas manuales, lo que orientó el esfuerzo de calidad de las últimas semanas.

Finalmente, la relación entre costo y arquitectura fue una conclusión transversal. El modelo serverless y los servicios gestionados no solo redujeron el costo inicial a cero, sino que también disminuyeron la carga operativa del equipo: no fue necesario administrar servidores, aplicar parches de sistema operativo ni configurar respaldos manuales. Este ahorro de tiempo se reinvirtió directamente en funcionalidades y en seguridad. La contrapartida es una mayor dependencia de proveedores externos, que el equipo considera aceptable para la etapa actual siempre que se mantengan las medidas de portabilidad descritas en este informe.

La última iteración dejó una lección sobre la experiencia de usuario: una funcionalidad correcta puede percibirse como defectuosa si no comunica lo que está pasando. El pago siempre funcionó, pero la falta de una pantalla de progreso hacía pensar al usuario que el sistema se había quedado pegado. Del mismo modo, mostrar en el historial las reservas que nunca se pagaron (por saldo insuficiente) generaba ruido. Escuchar esa retroalimentación y priorizar cambios puntuales, en lugar de rediseñar toda la interfaz, permitió mejorar la confianza en el producto sin poner en riesgo lo que ya estaba validado.

Como limitaciones, el prototipo opera en el ambiente de integración de Transbank y en planes gratuitos, por lo que aún no se han medido el rendimiento ni la disponibilidad bajo carga real. Quedan pendientes completar los datos legales de la empresa en los términos y contratos, definir los plazos de retención de los documentos de identidad, y registrar las mediciones de rendimiento sobre el despliegue final. Estas tareas están incorporadas en el plan de acción hacia la entrega final.

Como trabajo futuro, el equipo identifica cuatro líneas: obtener la afiliación comercial con Transbank para operar con pagos reales; migrar a los planes Pro de Supabase y Vercel con un dominio propio; incorporar pruebas de extremo a extremo y de carga para validar los criterios de rendimiento; y evaluar una verificación de identidad asistida que cumpla la nueva ley de datos personales, por ejemplo, procesando la biometría con consentimiento explícito y retención mínima.

En síntesis, el proyecto cumplió el objetivo de la unidad: Spotly cuenta con un diseño técnico documentado, una selección de tecnologías y proveedores justificada con evidencia, un prototipo que implementa los procesos centrales del negocio con controles de seguridad reales, criterios de aceptación alineados con la normativa chilena y estándares internacionales, y una planificación revisada que conduce a la entrega final. El equipo considera que la solución es técnicamente viable y que la base construida permite escalarla hacia un producto comercial.

Referencias

Biblioteca del Congreso Nacional de Chile. (1982). *Ley 18.101: Fija normas especiales sobre arrendamiento de predios urbanos*. https://www.bcn.cl/leychile/navegar?idNorma=29526

Biblioteca del Congreso Nacional de Chile. (1997). *Ley 19.496: Establece normas sobre protección de los derechos de los consumidores*. https://www.bcn.cl/leychile/navegar?idNorma=61438

Biblioteca del Congreso Nacional de Chile. (1999). *Ley 19.628: Sobre protección de la vida privada*. https://www.bcn.cl/leychile/navegar?idNorma=141599

Biblioteca del Congreso Nacional de Chile. (2002). *Ley 19.799: Sobre documentos electrónicos, firma electrónica y servicios de certificación de dicha firma*. https://www.bcn.cl/leychile/navegar?idNorma=196640

Biblioteca del Congreso Nacional de Chile. (2024). *Ley 21.719: Regula la protección y el tratamiento de los datos personales y crea la Agencia de Protección de Datos Personales*. https://www.bcn.cl/leychile

GitHub. (2026). *GitHub Actions documentation*. https://docs.github.com/actions

Google. (2024). *Web Vitals*. https://web.dev/articles/vitals

International Organization for Standardization. (2022). *ISO/IEC 27001:2022 Information security, cybersecurity and privacy protection — Information security management systems — Requirements*. ISO.

International Organization for Standardization. (2023). *ISO/IEC 25010:2023 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. ISO.

Anime.js. (2026). *Anime.js v4 documentation*. https://animejs.com/documentation

Meta Open Source. (2026). *React documentation*. https://react.dev

Motion. (2026). *Motion for React documentation*. https://motion.dev/docs/react

OWASP Foundation. (2021). *OWASP Top 10: 2021*. https://owasp.org/Top10/

PCI Security Standards Council. (2022). *Payment Card Industry Data Security Standard v4.0*. https://www.pcisecuritystandards.org

PostgreSQL Global Development Group. (2026). *PostgreSQL documentation: Exclusion constraints and row security policies*. https://www.postgresql.org/docs/

Pressman, R. S., & Maxim, B. R. (2021). *Ingeniería del software: un enfoque práctico* (9.ª ed.). McGraw-Hill.

Project Management Institute. (2021). *Guía de los fundamentos para la dirección de proyectos (Guía del PMBOK)* (7.ª ed.). PMI.

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

Supabase. (2026). *Pricing & fees*. https://supabase.com/pricing

Supabase. (2026). *Row Level Security*. https://supabase.com/docs/guides/database/postgres/row-level-security

Transbank. (2026). *Transbank Developers: Webpay Plus y Oneclick Mall*. https://www.transbankdevelopers.cl/documentacion/webpay-plus

Transbank. (2026). *Webpay: tarifas y condiciones comerciales*. https://www.transbank.cl

Vercel. (2026). *Pricing*. https://vercel.com/pricing

Vercel. (2026). *Cron jobs*. https://vercel.com/docs/cron-jobs

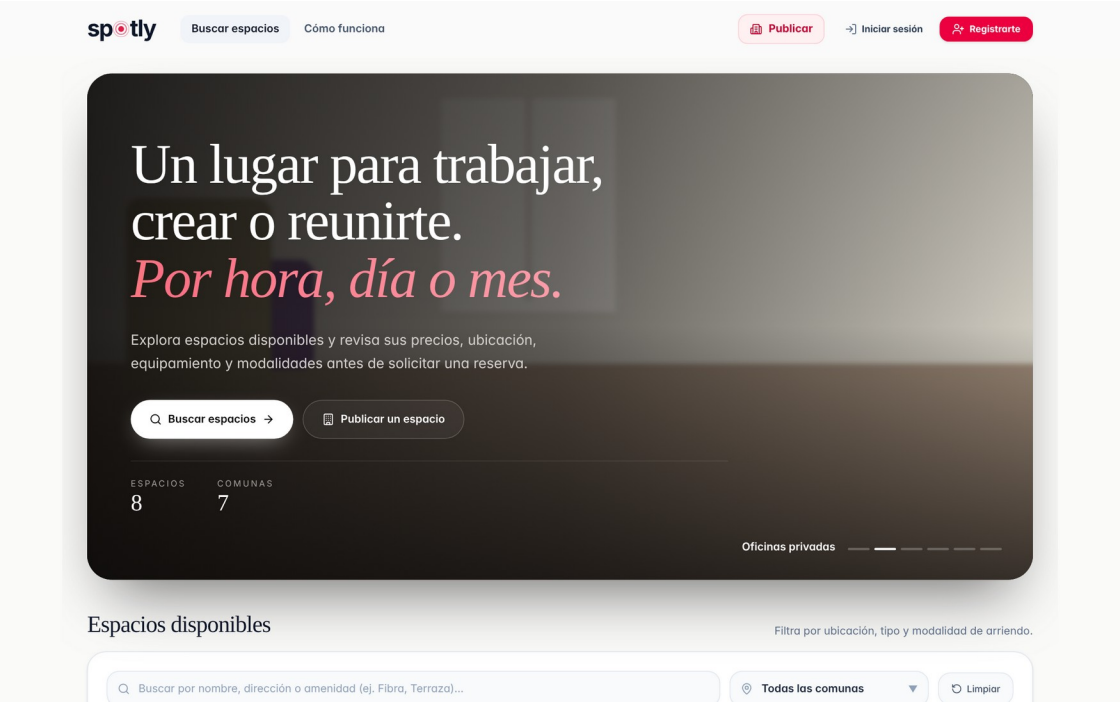
Vitest. (2026). *Vitest documentation*. https://vitest.dev

World Wide Web Consortium. (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. https://www.w3.org/TR/WCAG21/

Anexo A. Capturas del prototipo

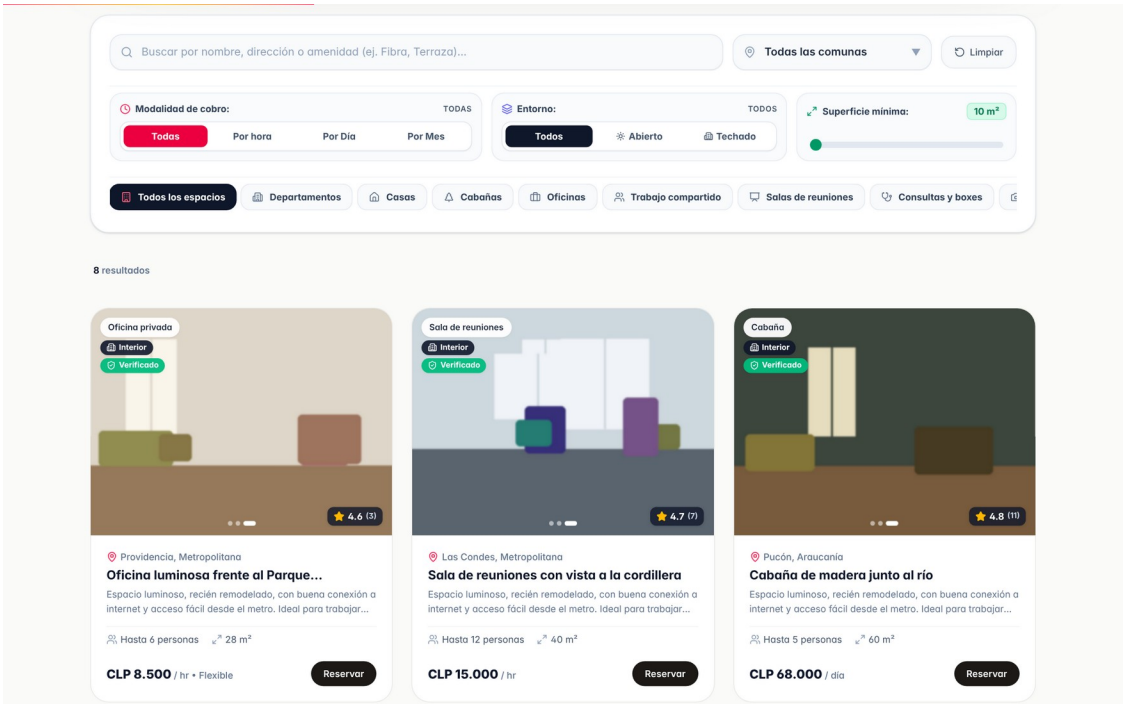
Las siguientes capturas muestran las vistas principales del prototipo en su versión actual, tomadas con un navegador automatizado sobre datos de demostración. El Manual de usuario contiene el detalle paso a paso de cada una.

Figura 7
Página de inicio con portada de fotografías y logotipo



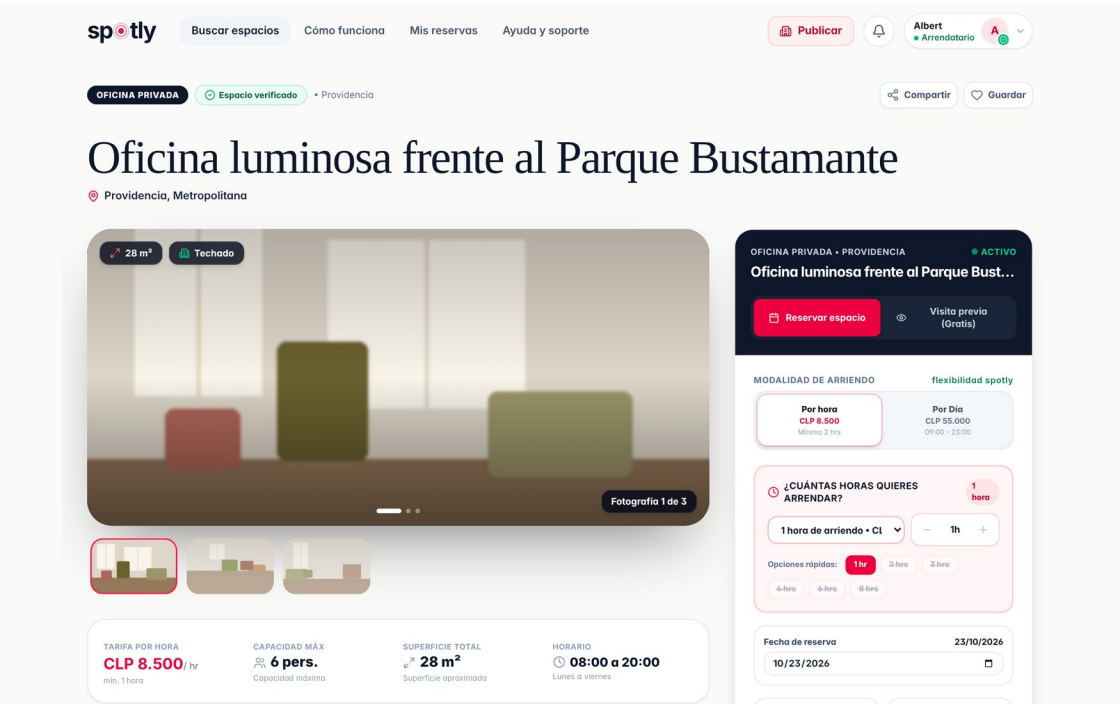
Nota. Captura del prototipo Spotly con datos de demostración.

Figura 8
Búsqueda de espacios con filtros



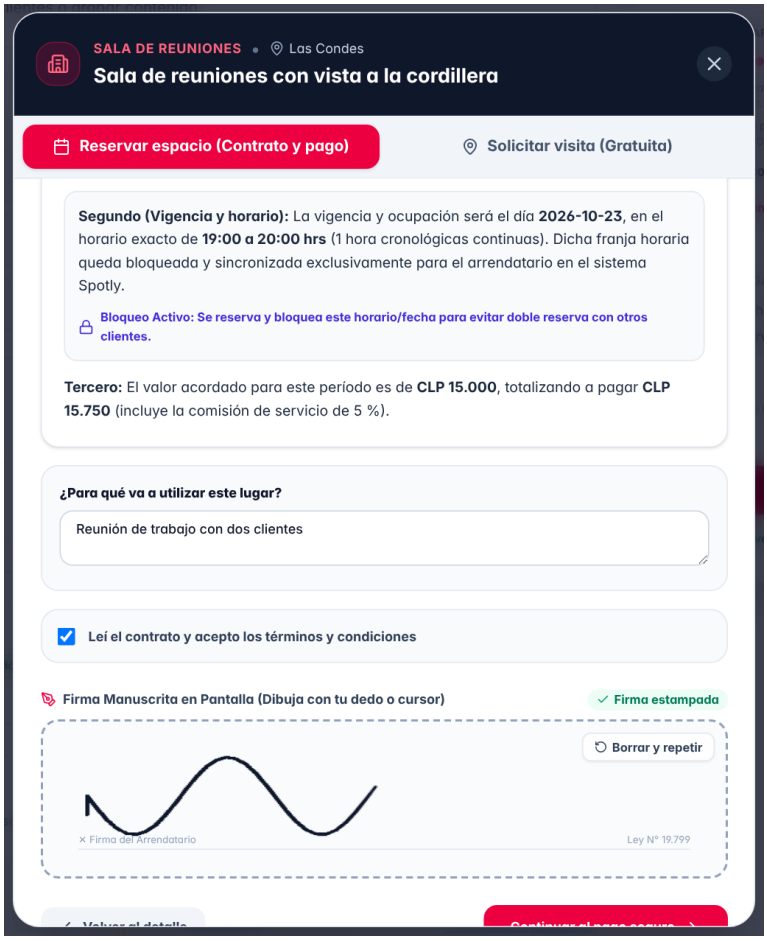
Nota. Captura del prototipo Spotly con datos de demostración.

Figura 9
Detalle del espacio con galería y panel de reserva



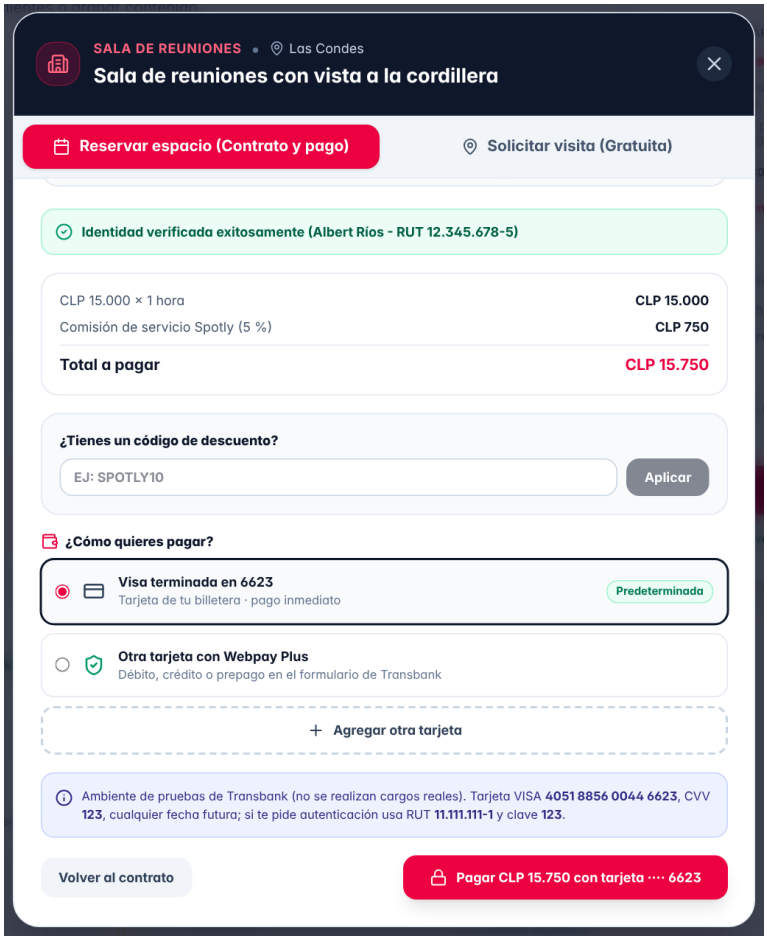
Nota. Captura del prototipo Spotly con datos de demostración.

Figura 10
Contrato de arrendamiento y firma electrónica



Nota. Captura del prototipo Spotly con datos de demostración.

Figura 11
Selección del medio de pago (billetera o Webpay Plus)



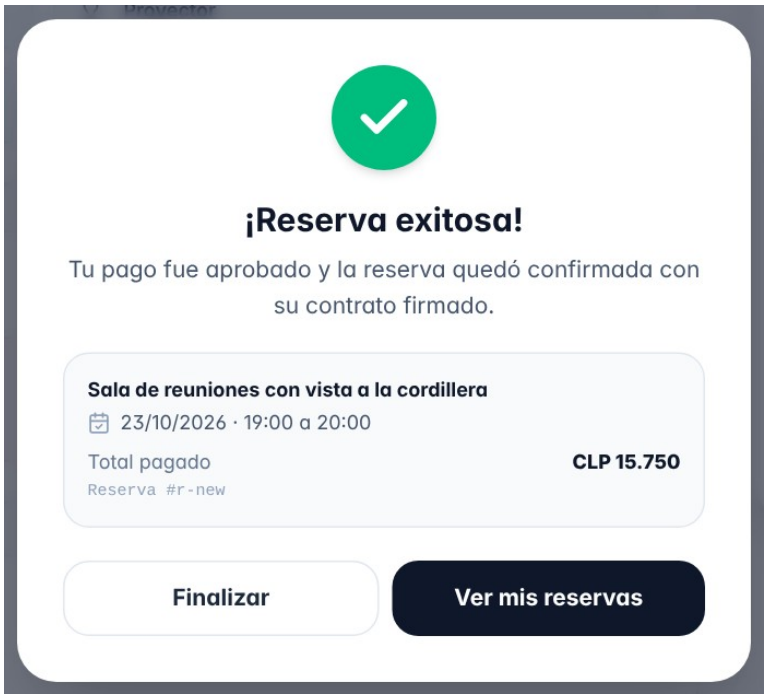
Nota. Captura del prototipo Spotly con datos de demostración.

Figura 12
Pantalla de progreso: verificando y realizando la reserva



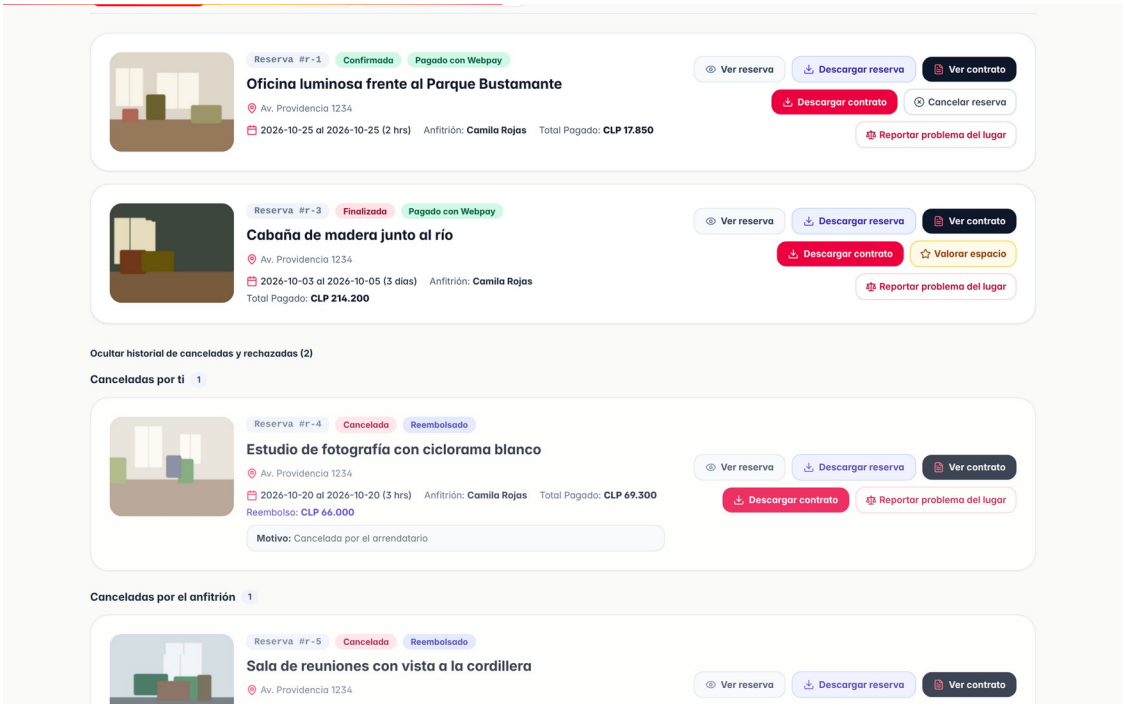
Nota. Captura del prototipo Spotly con datos de demostración.

Figura 13
Resultado de la reserva



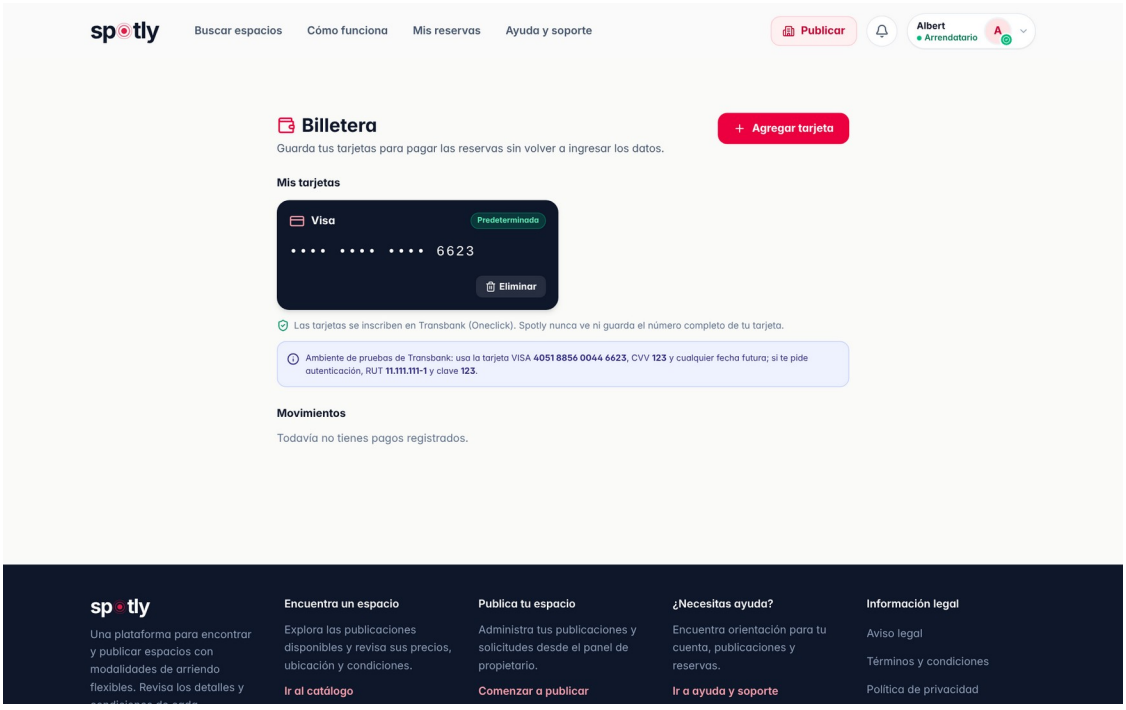
Nota. Captura del prototipo Spotly con datos de demostración.

Figura 14
Mis reservas: historial de canceladas separado por quién canceló



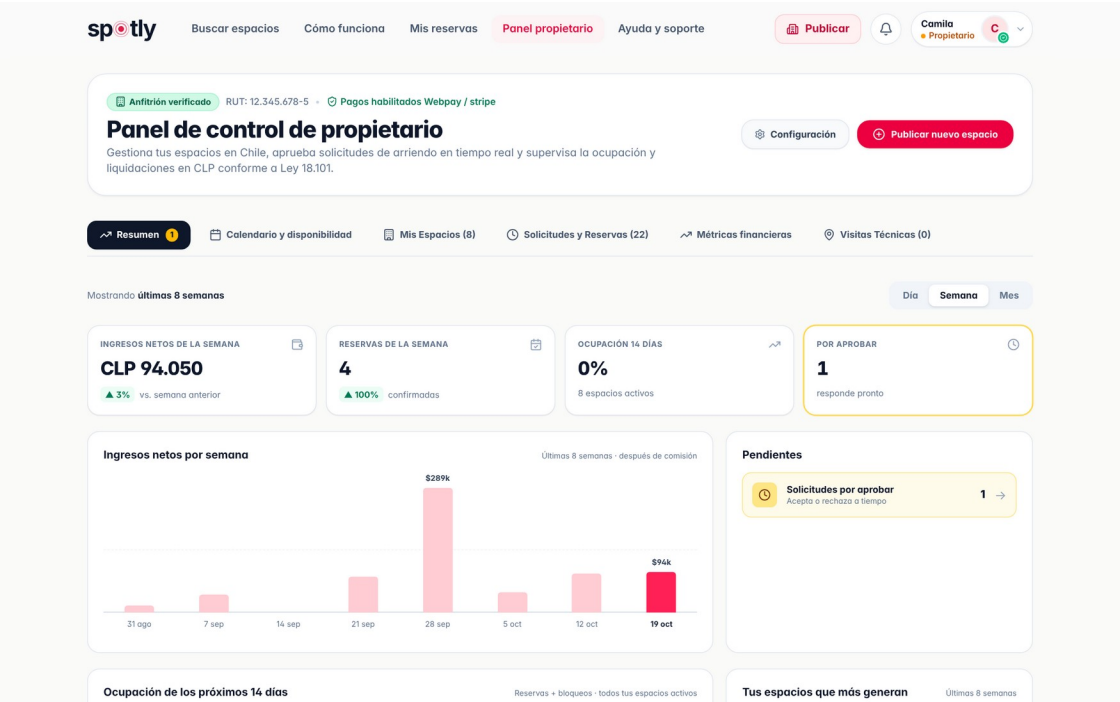
Nota. Captura del prototipo Spotly con datos de demostración.

Figura 15
Billetera de tarjetas (Transbank Oneclick)



Nota. Captura del prototipo Spotly con datos de demostración.

Figura 16
Panel del propietario con vista por semana



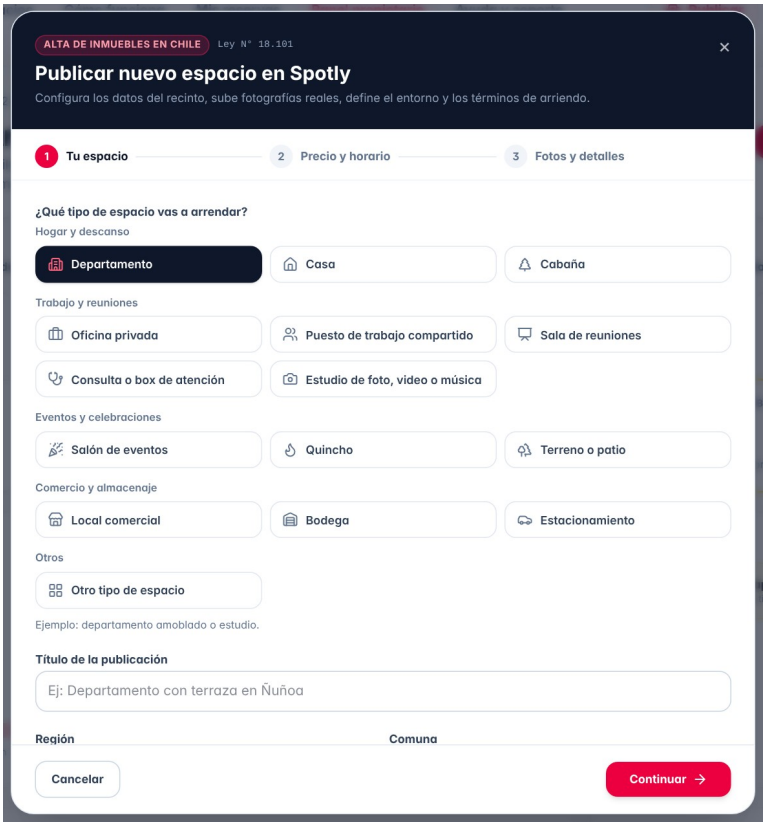
Nota. Captura del prototipo Spotly con datos de demostración.

Figura 17
Cancelación de una reserva por el propietario



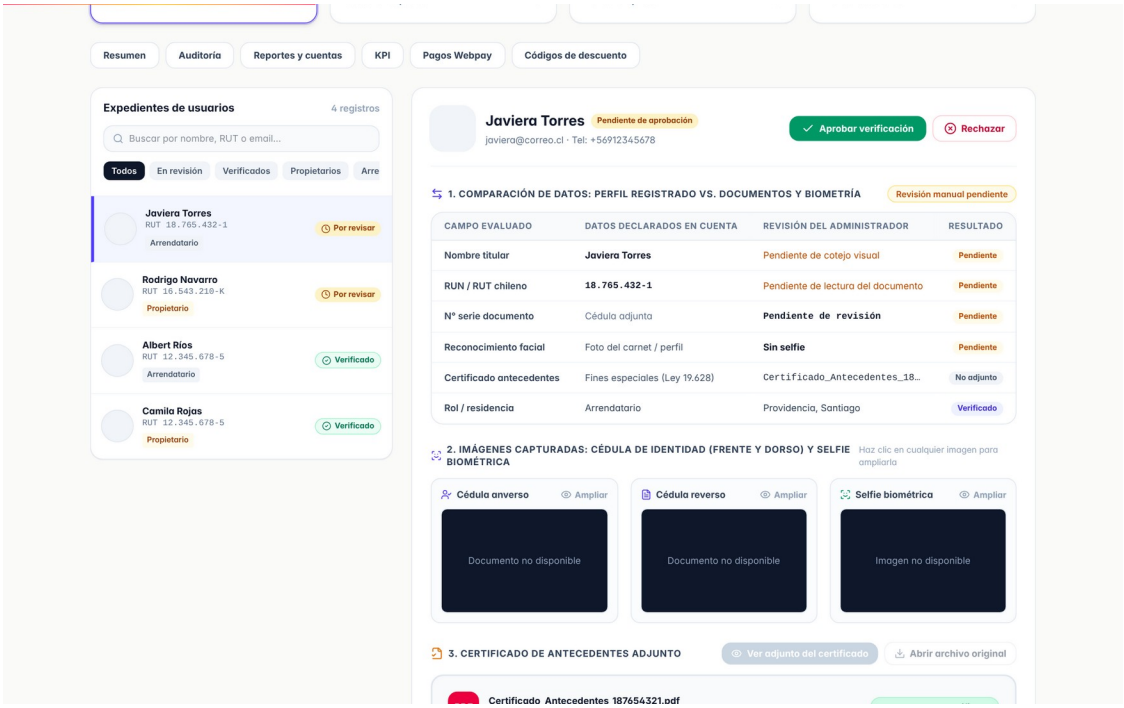
Nota. Captura del prototipo Spotly con datos de demostración.

Figura 18
Publicar espacio: asistente de 3 pasos



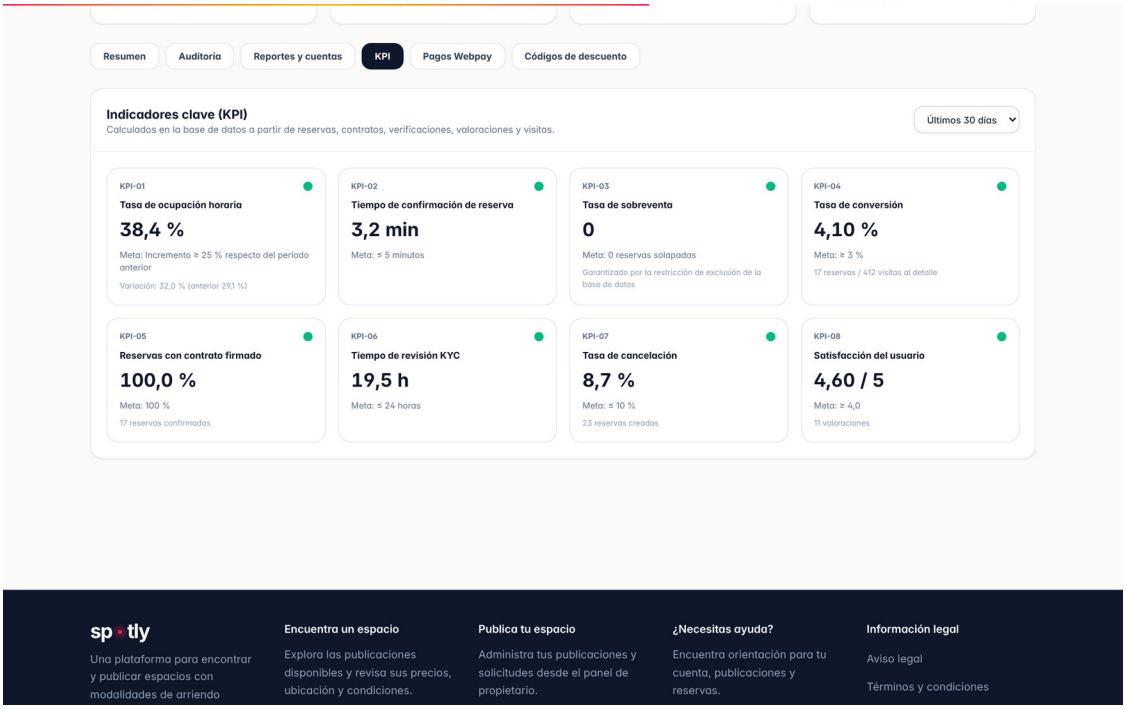
Nota. Captura del prototipo Spotly con datos de demostración.

Figura 19
Administración: revisión de verificación de identidad



Nota. Captura del prototipo Spotly con datos de demostración.

Figura 20
Administración: indicadores clave (KPI)



Nota. Captura del prototipo Spotly con datos de demostración.